Počítačová grafika

Sledování paprsku

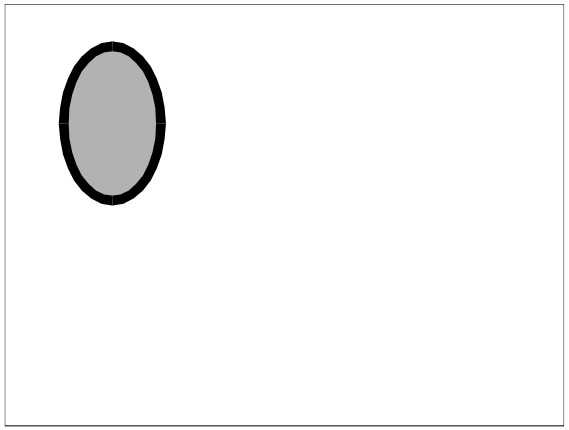
prof. Ing. **Adam Herout**, PhD.

BRNO FACULTY
UNIVERSITY OF INFORMATION
OF TECHNOLOGY TECHNOLOGY

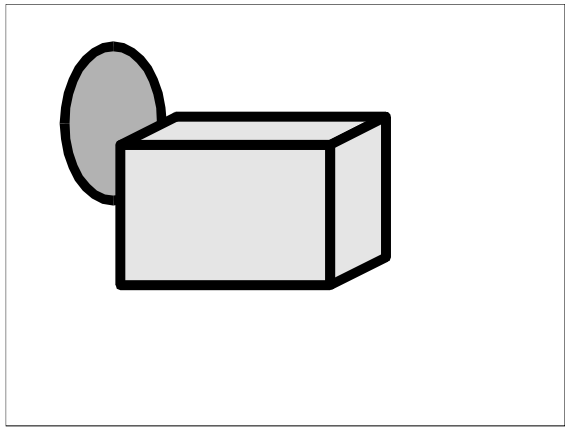
Cíl: Fotorealistická grafika

Zobrazovací metody – přímé zobrazování

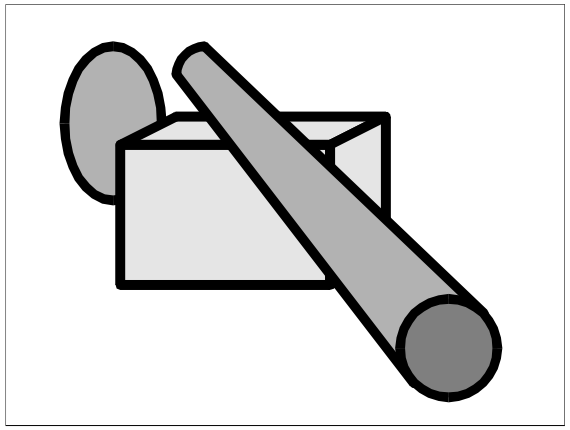
- Objekty jsou zpracovávány sekvenčně
- Není interakce mezi objekty – bez stínů



1.



2.

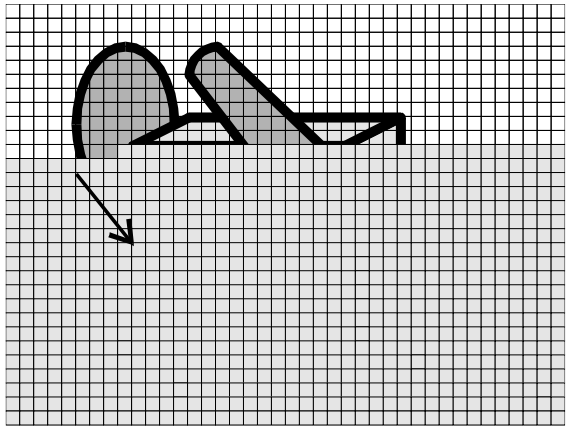


3.

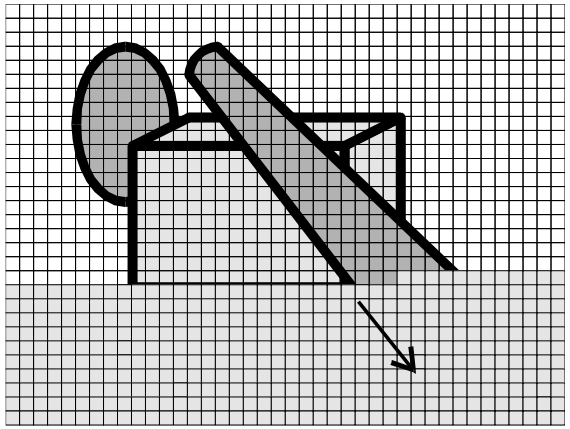
- (Object order)

Zobrazovací metody – po pixelech

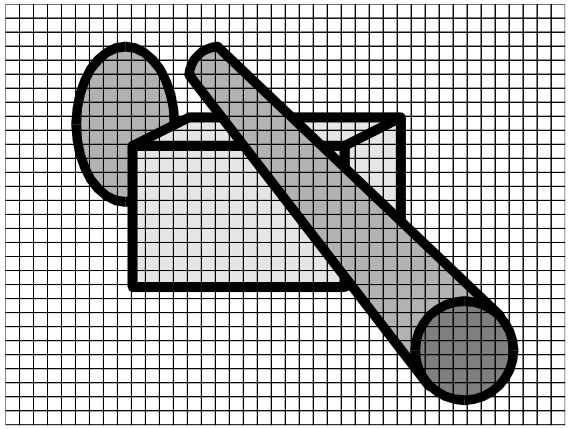
- Pixely se zpracovávají sekvenčně
- Není globální interakce – bez měkkých stínů



1.



2.

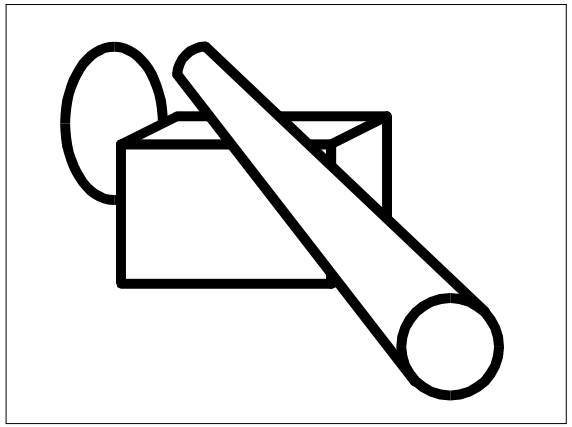


3.

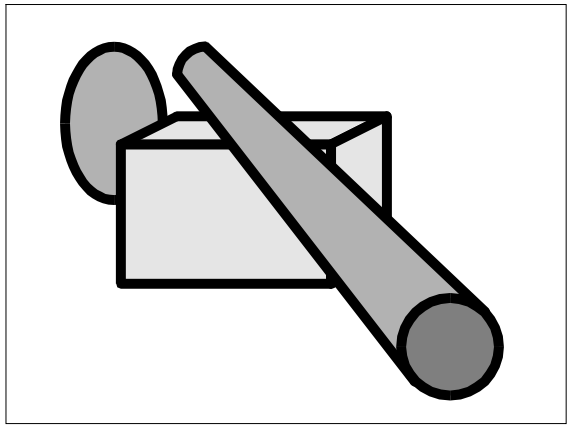
- (Image order)

Zobrazovací metody – celá scéna (globální)

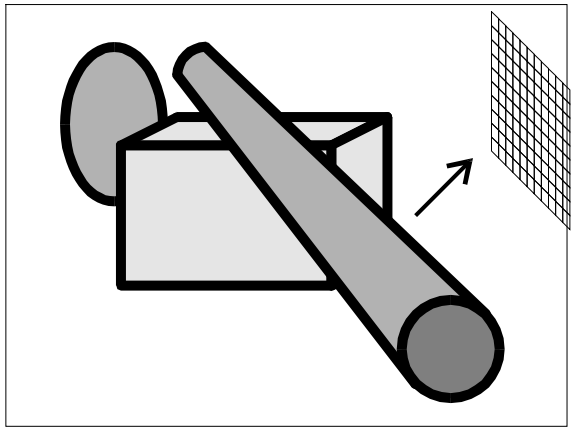
- Zpracovává se celá scéna najednou
- Velmi složité – kvůli realizaci bez „tvrdých stínů“



1.



2.

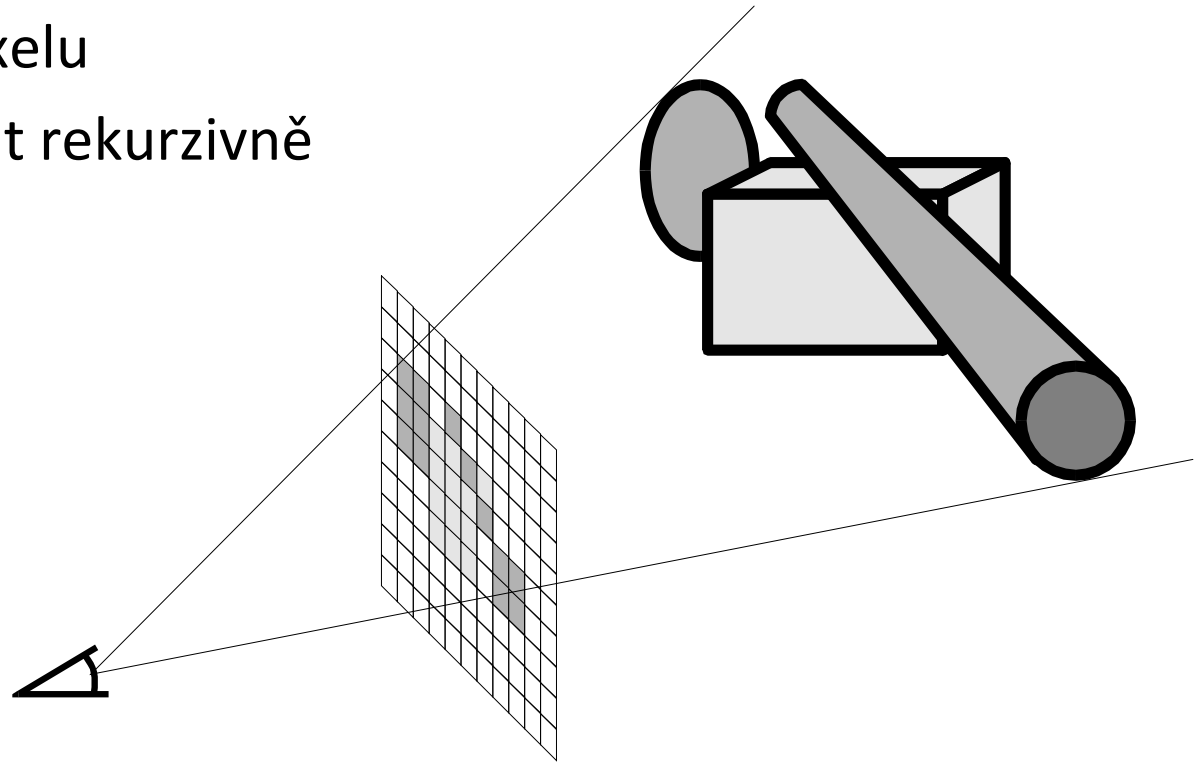


3.

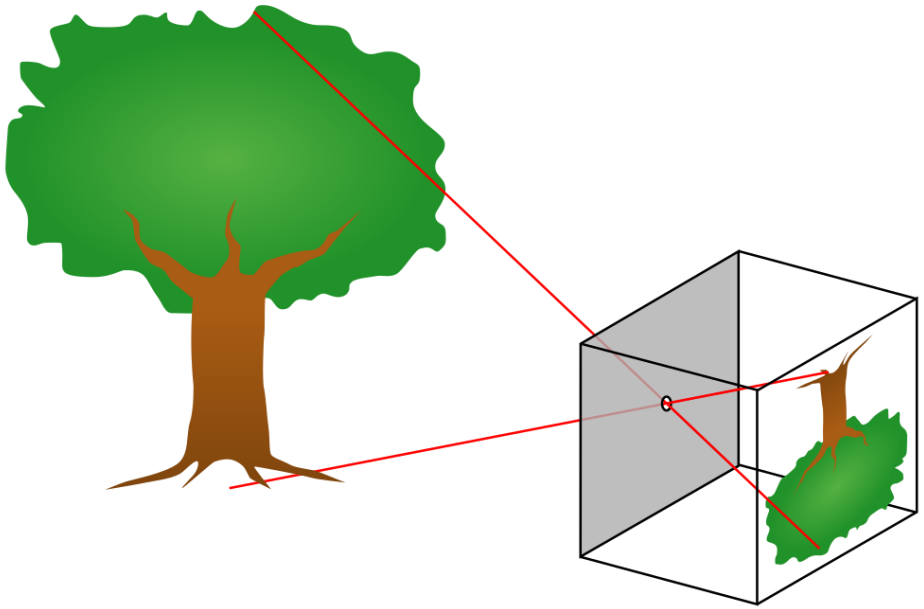
- (Image order + fyzikální simulace šíření světla)

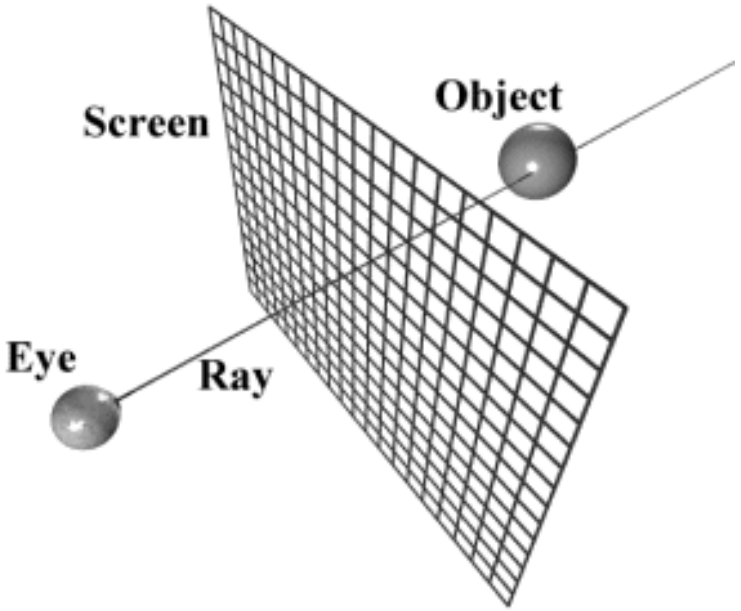
Princip vrhání paprsku (ray casting)

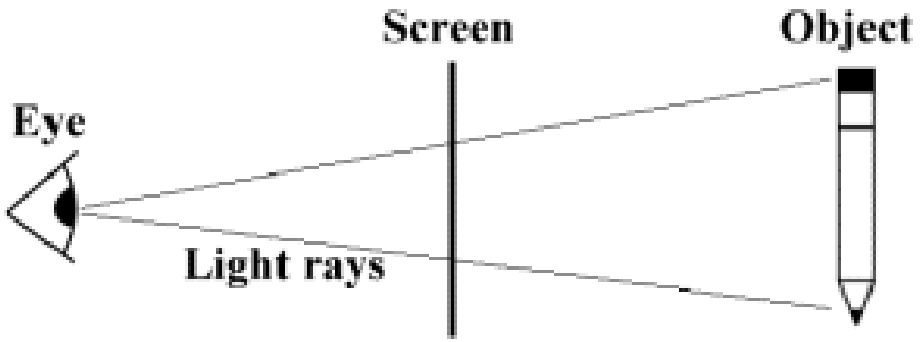
- Pro každý pixel
  - Poslat paprsek oko – pixel – scéna
  - Spočítat, co je vidět
  - Vyhodnotit hodnotu (barvu) pixelu
  - Pokud je to potřeba, pokračovat rekurzivně



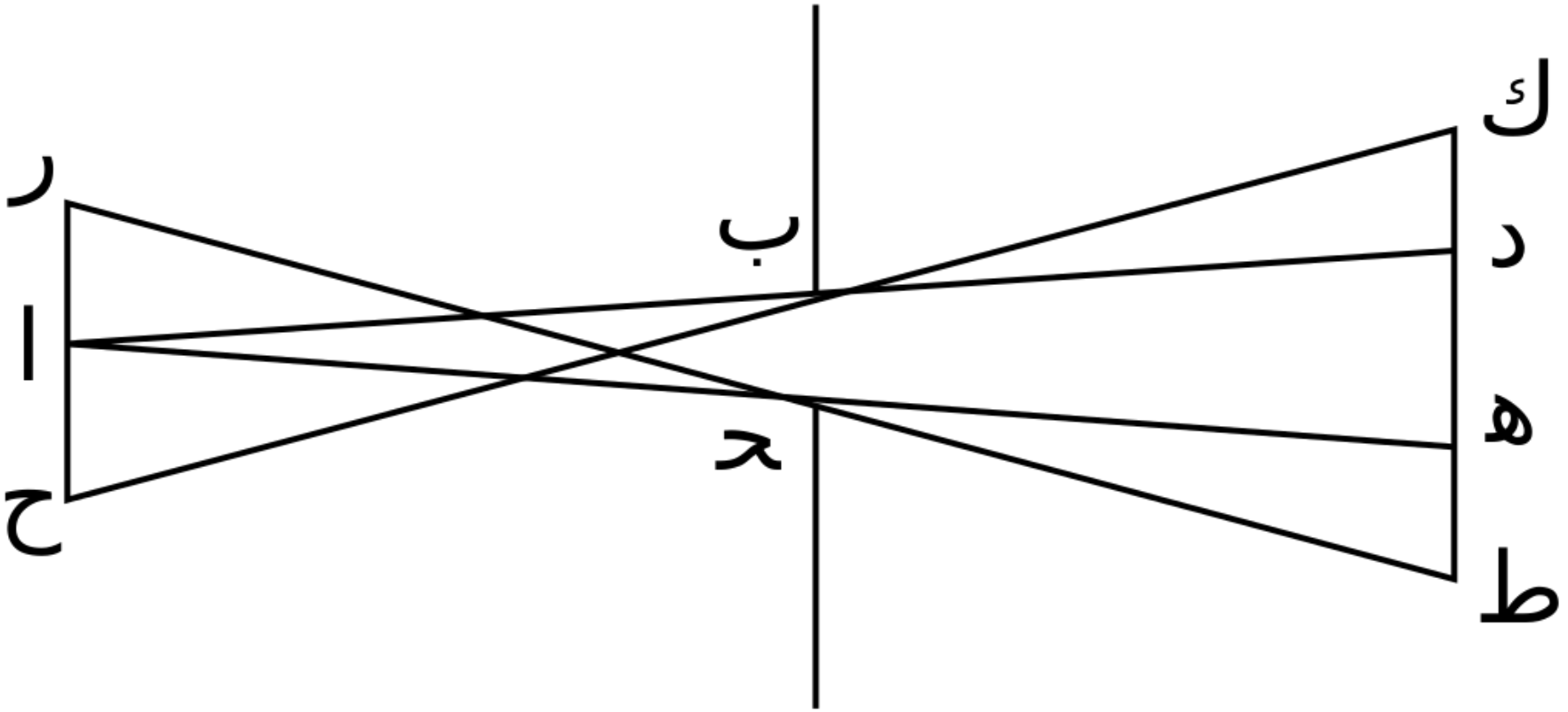
Princip vrhání paprsku







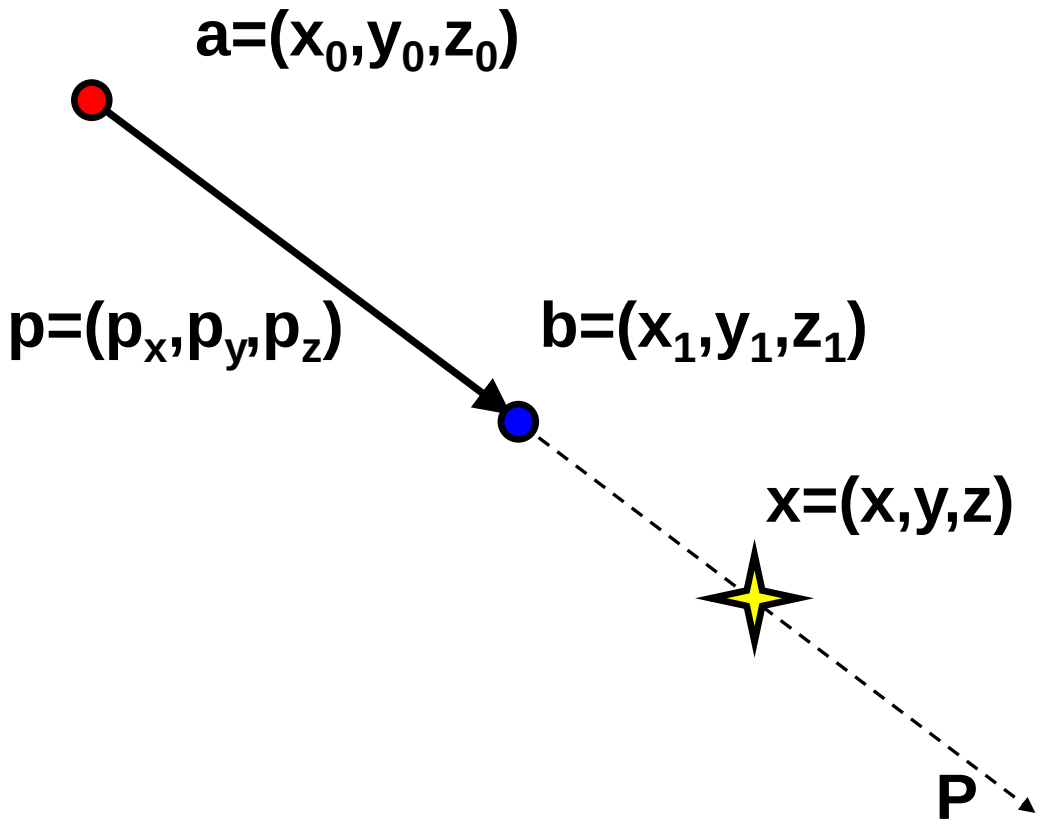
Ibn al-Haytham, 10. století



Ray casting v 15. století

Trocha matematiky – paprsek

paprsek = polopřímka



$$\vec{p} = \vec{b} - \vec{a}$$

$$P = \{\vec{x} \mid \exists t \in <0, \infty); \vec{x} = \vec{a} + \mathbf{t}\vec{p}\}$$

Trocha programování – vektor

```
struct Vector {
  double x,y,z;
  Vector() { x = 0; y = 0; z = 0; }       // impl. konstruktor
  Vector(double x, double y, double z) { // init. konstruktor
    this->x = x; this->y = y; this->z = z;
  }
  inline double Length() const {          // zjištění délky
    return sqrt(x*x + y*y + z*z);
  }
  Vector &Normalize() {                   // normalizace
    double length = Length();             // (bude jednotkový)
    x /= length; y /= length; z /= length; return *this;
  }
  Vector &operator += (const Vector &a) { // přičtení vektoru
    x += a.x; y += a.y; z += a.z;
    return *this;
  }
};
```

Trocha programování – vektor

```
inline Vector operator + (const Vector &a, const Vector &b) {
  return Vector(a.x + b.x, a.y + b.y, a.z + b.z);
}                                          // součet vektorů
inline Vector operator - (const Vector &a, const Vector &b) {
  return Vector(a.x - b.x, a.y - b.y, a.z - b.z);
}                                          // rozdíl vektorů
inline double operator * (const Vector &a, const Vector &b) {
  return a.x*b.x + a.y*b.y + a.z*b.z; // skalární součin
}
inline Vector operator * (double k, const Vector &a) {
  return Vector(k*a.x, k*a.y, k*a.z); // změna délky
}
inline Vector cross(const Vector &a, const Vector &b) {
  return Vector(a.y*b.z - a.z*b.y,     // vektorový součin
                a.z*b.x - a.x*b.z,
                a.x*b.y - a.y*b.x);
}
inline std::ostream & operator << (std::ostream &str,
                                   const Vector &a) {
  return str << "< " << a.x << " " << a.y << " " << a.z << " >";
}                                          // textový výpis (pro ladění)
```

Trocha programování – paprsek

```
class Ray {
  Vector a;     // začátek paprsku - bod
  Vector p;     // směr - vektor
public:
  Ray(const Vector &start, const Vector &direction) {
    a = start; p = direction;
  }
  inline const Vector &Start() const { return a; }
  inline const Vector &Direction() const { return p; }
  inline Vector GetPoint(double t) const {
    return a + t*p;                  // zjistí bod určený
  }                                  // relativní polohou na pap. void ShiftStart(double
   shiftby = 1e-6) {
    a += shiftby*p;                  // posune začátek
  }
};
```

Trocha matematiky – průsečík s koulí

$$K = \{\vec{x}; (\vec{x} - \vec{s})^2 - r^2 < 0\}$$

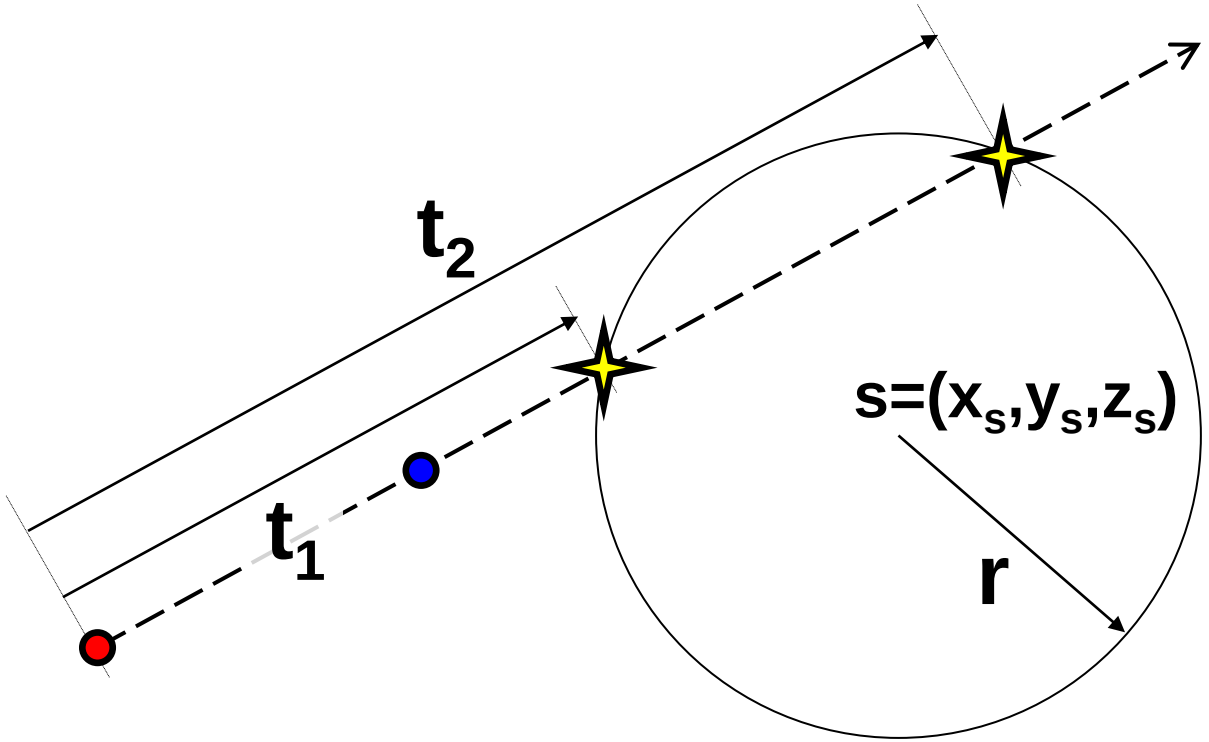
$$(\vec{a} + \mathbf{t}\vec{p} - \vec{s})^2 - r^2 = 0$$

$$\mathbf{t}^2 \vec{p}^2 + 2\mathbf{t}\vec{p}(\vec{a} - \vec{s}) + (\vec{a} - \vec{s})^2 - r^2 = 0$$

$$x = \vec{a} + t_1 \vec{p} \quad nebo$$

$$x = \vec{a} + t_2 \vec{p}$$

$$\vec{n} = \vec{x} - \vec{s}$$



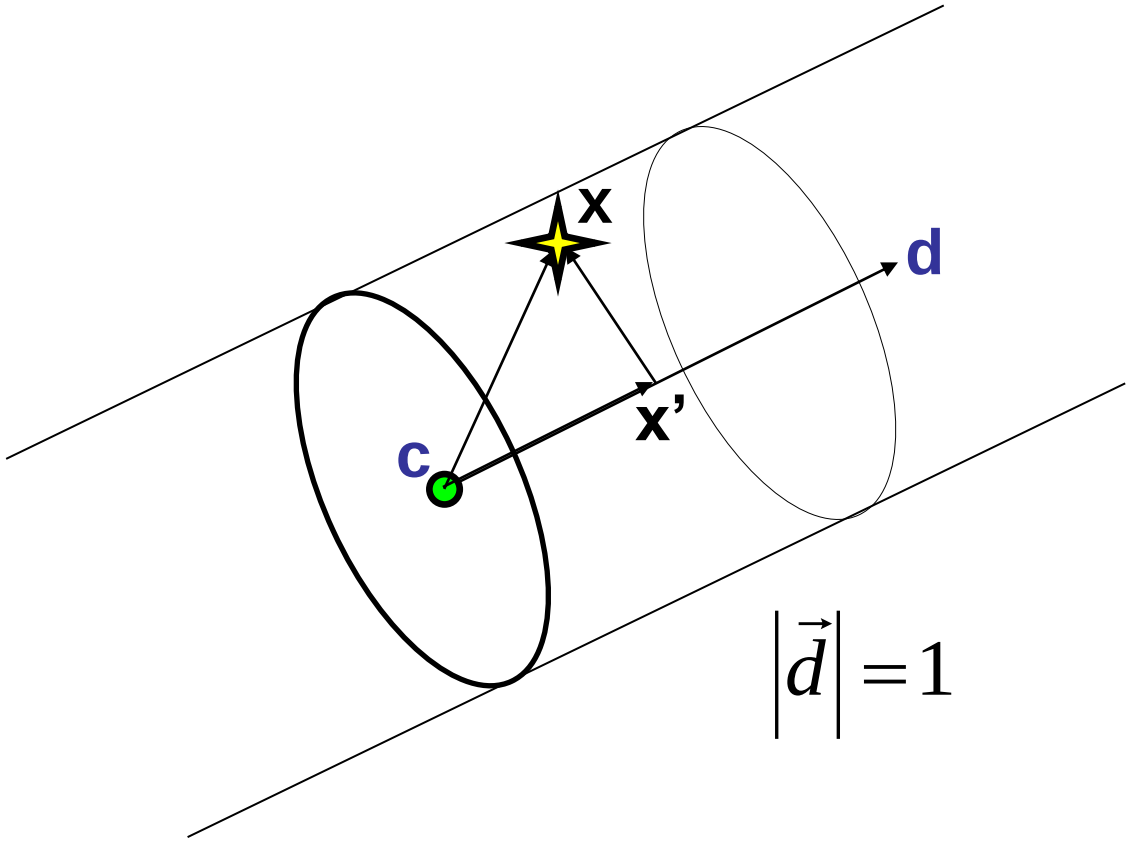
Trocha programování – koule

```
class Sphere : public Shape {
  Vector c; double r;
public:
  Sphere(const Vector &centre, double radius) { c = centre; r = radius; }
  virtual Intersection IntersectFirst(const Ray &ray) {
    const Vector &a = ray.Start();
    const Vector &p = ray.Direction();
    double aa = p*p;
    double bb = 2* (p * (a - c));
    double cc = (a-c)*(a-c) - r*r;
    double D = bb*bb - 4*aa*cc;
    if (D > 0) {
      double sD = sqrt(D);
      double t1 = (-bb - sD) / (2*aa);
      if (t1 > 0) return Intersection(Intersection::ikInto, this, t1);
      double t2 = (-bb + sD) / (2*aa);
      if (t2 > 0) return Intersection(Intersection::ikOutfrom, this, t2);
    }
    return Intersection(Intersection::ikNone);
  }
  virtual Vector GetNormal(const Vector &position) {
    Vector n = position - c;  n.Normalize();
    return n;
  }
};
```

Trocha matematiky – průsečík s válcem

$$V = \{\vec{x} \mid (\vec{x} - \vec{x}')^2 = r^2\}$$

$$\vec{x}' = \vec{c} + (\vec{x} \cdot \vec{d})\vec{d}$$



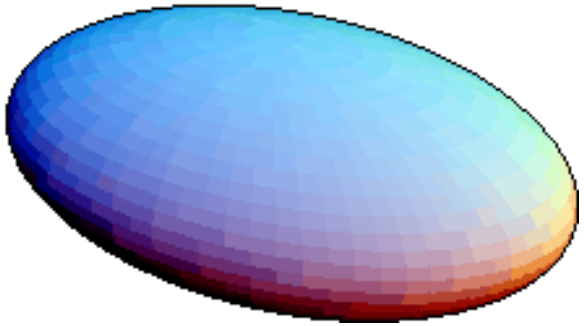
$$\left|\vec{d}\right| = 1$$

Trocha programování – válec

```
class Cylinder : public Shape {
  Vector p,d;  double r;
public:
  Cylinder(const Vector &centre, const Vector &direction, double radius) {
    p = centre; d = direction; d.Normalize(); r = radius;
  }
  virtual Ranges<Shape> Intersect(const Ray &ray) {
    Ranges<Shape> ranges;
    const Vector &c = ray.Start() - p;
    const Vector &b = ray.Direction();
    double bd = b * d;    double b2 = b * b;    double cb = c * b;
    double cd = c * d;    double c2 = c * c;
    double A = b2 - bd*bd;              double B = 2*cb - 2*cd*bd;
    double C = c2 - cd*cd - r*r;        double D = B*B - 4*A*C;
    if (D > 0) {
      double sD = sqrt(D);
      double t1 = (-B - sD) / (2*A);    double t2 = (-B + sD) / (2*A);
      ranges.Add(Range<Shape>(t1, t2, this, this));
    }
    return ranges;
  }
  virtual Vector GetNormal(const Vector &position) {
    Vector n = position - (p + ((position - p)*d) * d);    n.Normalize();
    return n;
  }
};
```

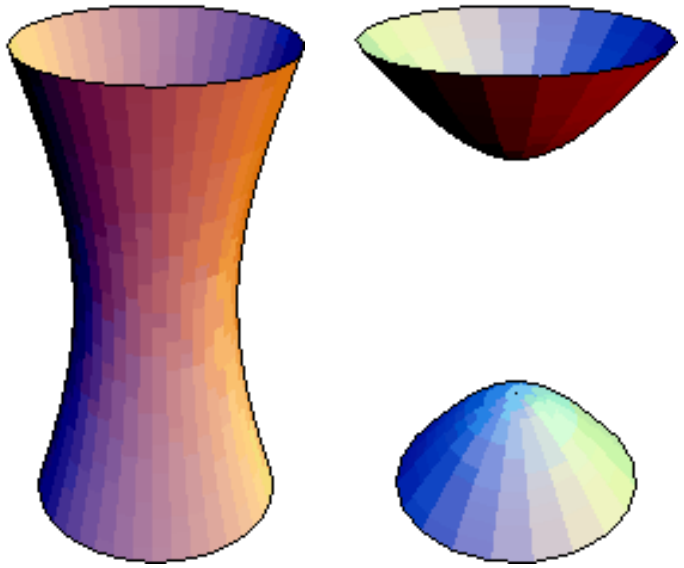
Další kvadratické objekty (Quadrics)

elipsoid

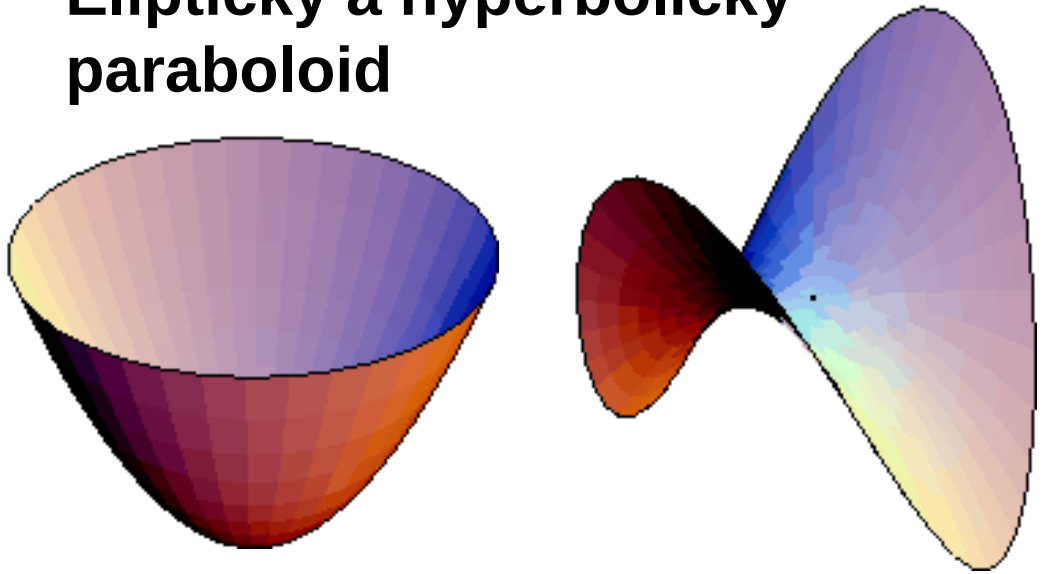


$$ax^2 + by^2 + cz^2 +$$
$$2fyz + 2gzx + 2hxy +$$
$$2px + 2qy + 2rz + d = 0$$

hyperboloid



Eliptický a hyperbolický paraboloid

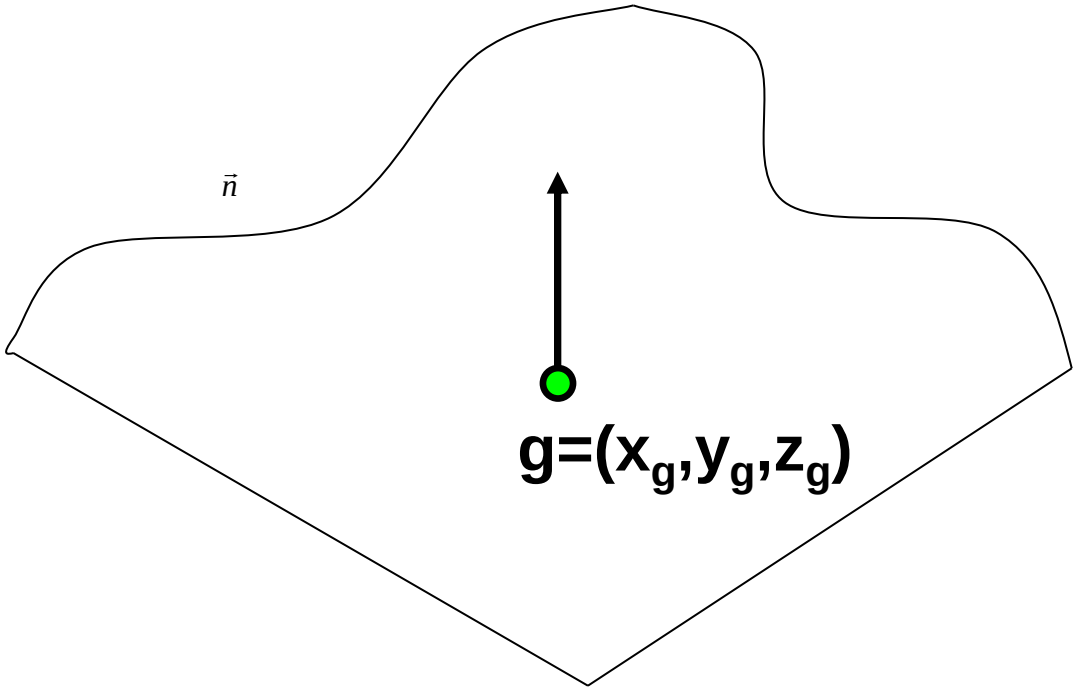


Trocha matematiky – určení roviny

$$R = \{\vec{x}; \vec{n}\vec{x} + d = 0\}$$

$$R = \{\vec{x}; \vec{n}\vec{x} + d < 0\}$$

d: dosazením $g=(x_g,y_g,z_g)$ do rovnice



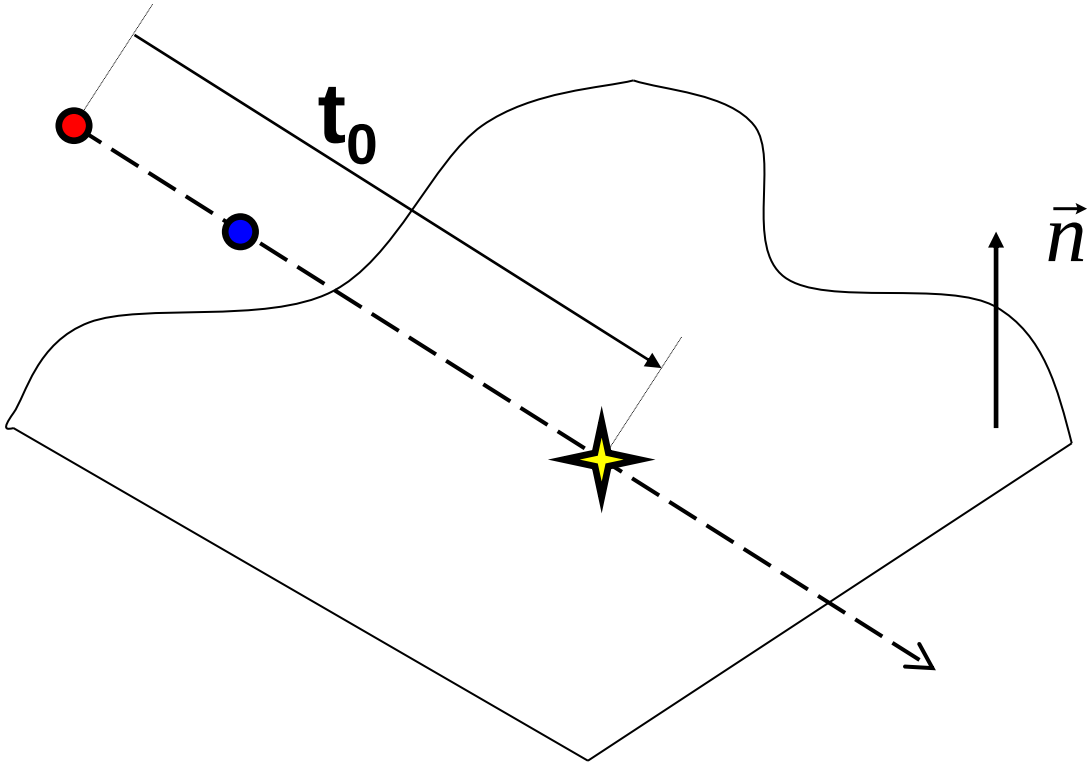
Trocha matematiky – průsečík s rovinou

$$R = \{\vec{x}; \vec{n}\vec{x} + d = 0\}$$

$$\vec{n}(\vec{a} + \mathbf{t}_0 \vec{p}) + d = 0$$

$$\mathbf{t}_0 \vec{n}\vec{p} + \vec{n}\vec{a} + d = 0$$

$$\vec{x}_0 = \vec{a} + \mathbf{t}_0 \vec{p}$$



Trocha programování – rovina

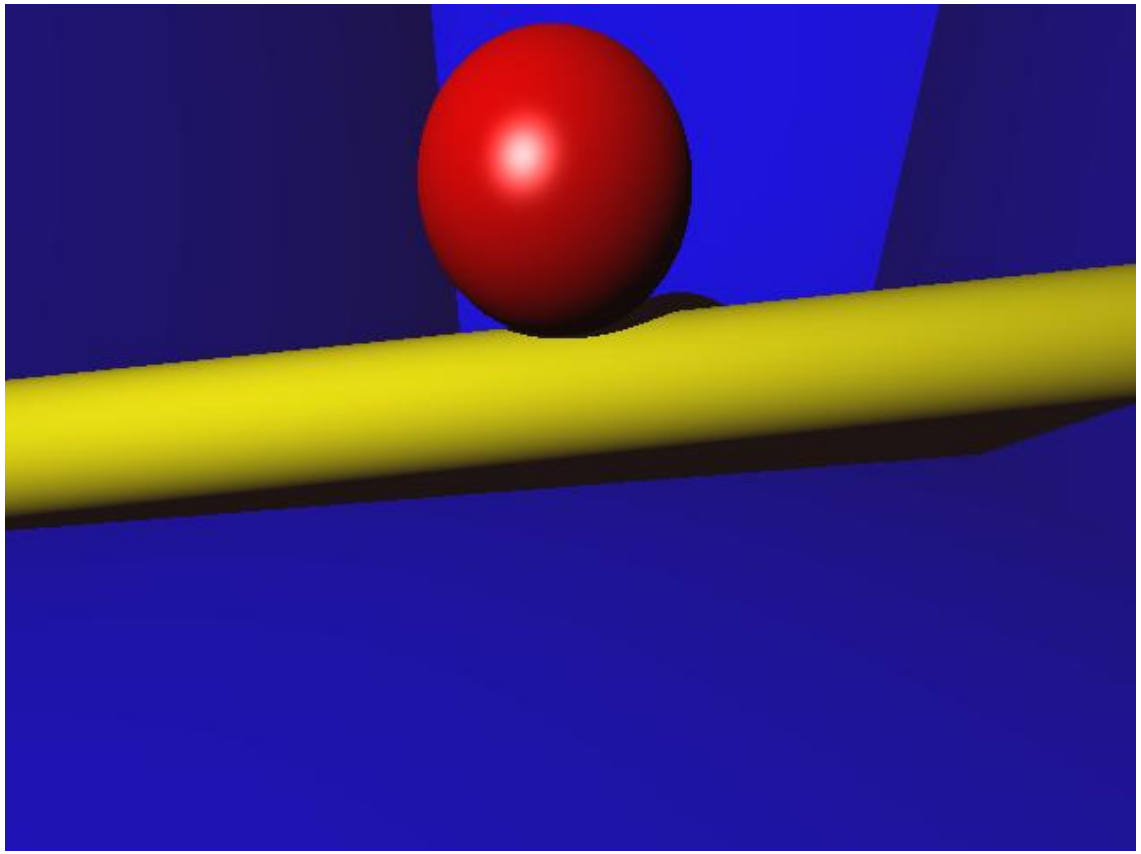
```
class Plane : public Shape {
  Vector n;  double d;
public:
  Plane(const Vector &normal, const Vector &point) {
    n = normal; n.Normalize(); d = -(n*point);
  }
  virtual Intersection IntersectFirst(const Ray &ray) {
    double np = n*ray.Direction();
    if (np == 0) return Intersection(Intersection::ikNone);
    double t = -(d + n*ray.Start()) / np;
    if (t > 0) {
      return Intersection((np > 0) ? Intersection::ikInto :
                                     Intersection::ikOutfrom,
                          this, t);
    } else return Intersection(Intersection::ikNone);
  }
  virtual Vector GetNormal(const Vector &position) {return n;}
};
```

Koule, válec, rovina

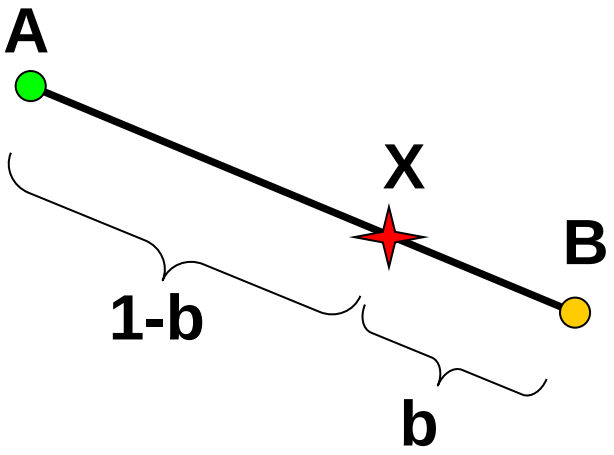
```
scene.Add(new ShapeObject(
 new Sphere(Vector(0, 4, 0), 2),
 matRed));

scene.Add(new ShapeObject(
 new Cylinder(Vector(0, 1, 0),
              Vector(1, 0.1, 0.3), 1.2),
 matYellow));

scene.Add(new ShapeObject(
 new Plane(Vector(0, 0, 1),
           Vector(0, 0, 15)), matBlue));
scene.Add(new ShapeObject(
 new Plane(Vector(0, 1, 0),
           Vector(0, -1.5, 0)), matBlue));
scene.Add(new ShapeObject(
 new Plane(Vector(1, 0, 0),
           Vector(-10, 0, 0)), matBlue));
scene.Add(new ShapeObject(
 new Plane(Vector(-1, 0, 0),
           Vector(10, 0, 0)), matBlue));
```

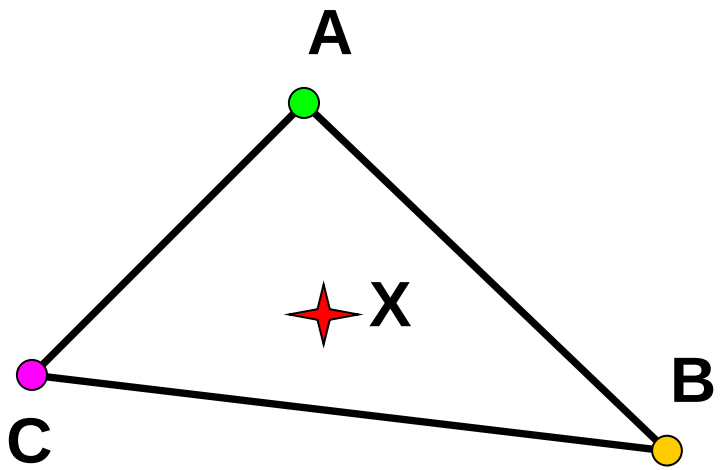


Barycentrické souřadnice



$$X = A + b(B-A)$$
$$X = aA + bB,\ a+b=1$$

$$X = aA + bB + cC$$
$$a+b+c = 1$$



Průsečík paprsku s trojúhelníkem

X = aA + bB + cC

X = (1 - b - c)A + bB + cC

X - A = (B-A)b + (C-A)c

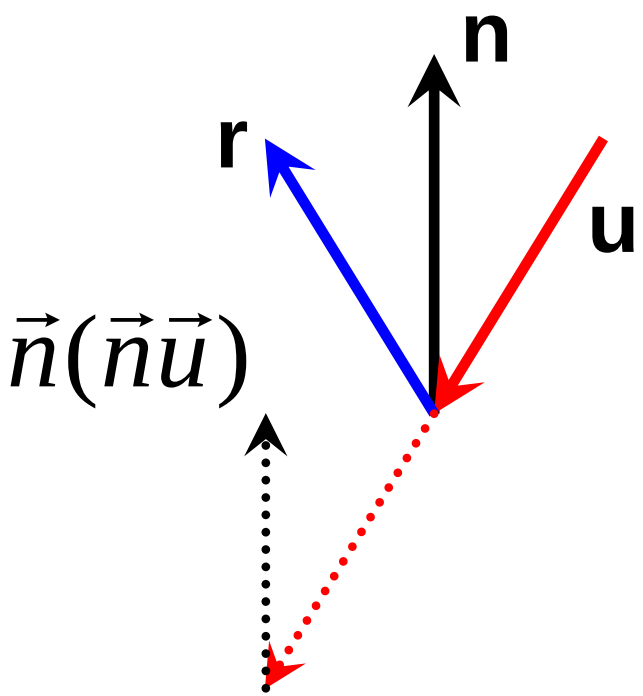
$$x_X - x_A = (x_B - x_A)b + (x_C - x_A)c$$

$$y_X - y_A = (y_B - y_A)b + (y_C - y_A)c$$

- Průsečík s rovinou, ve které trojúhelník leží
- Podle normálového vektoru trojúhelníka: „nejrovnoběžnější“ souřadnou rovinu
- Na ni promítnout trojúhelník a průsečík
- Zjistit barycentrické souřadnice průsečíku
- Když jsou mezi 0 a 1, průsečík leží v trojúhelníku

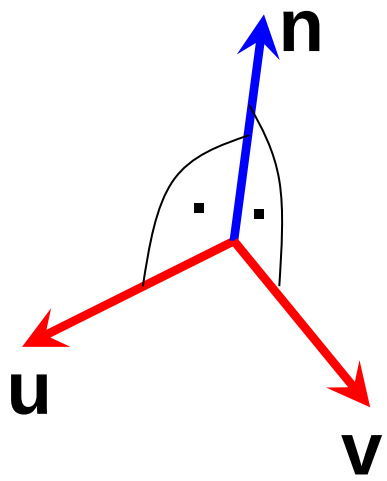
NatureSuxx – Federation Against Nature (demo)

Trocha matematiky – hry s vektory



$$\vec{r} = \vec{u} - 2\vec{n}(\vec{u}\vec{n})$$

$$\left|\vec{n}\right| = 1$$



$$\vec{n} = \vec{u} \times \vec{v}$$

$$\left|\vec{n}\right| = \left|\vec{u}\right|\left|\vec{v}\right| \sin\alpha$$

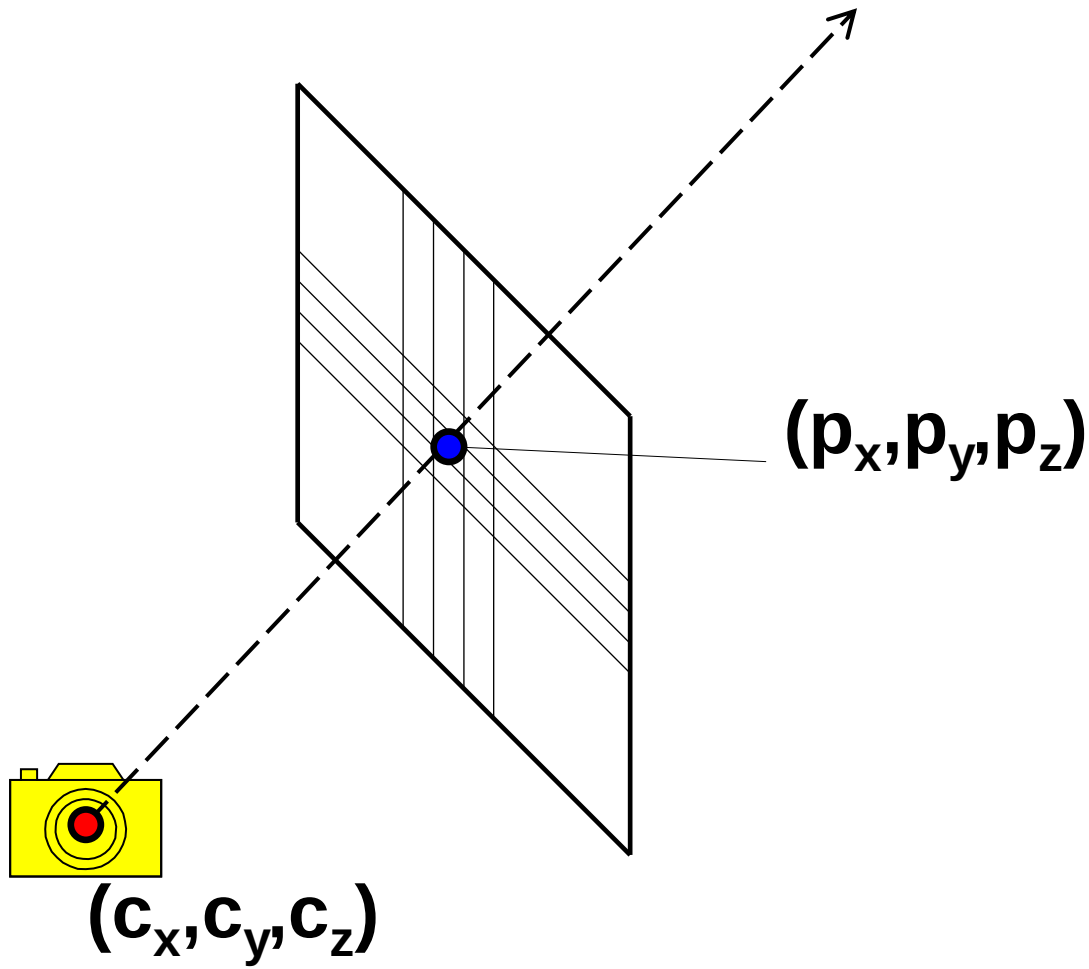
$$x_n = y_u z_v - z_u y_v$$

$$y_n = z_u x_v - x_u z_v$$

$$z_n = x_u y_v - y_u x_v$$

Vrhání paprsku

```
void Render(const Camera &camera,
            ObjectGroup &scene,
            LightSet &lights)
{
  int width = frame->GetWidth();
  int height = frame->GetHeight();
  for (int y = 0; y < height; y++) {
    for (int x = 0; x < width; x++) {
      Ray ray = camera.GetRay(2.0*x/width - 1,
                              2.0*y/height - 1);
      Color c = TraceRay(ray, scene, lights, 5);
      frame->PutPixel(x, y, c);
    }
  }
}
```



Vrhání paprsku

```
Color TraceRay(const Ray &ray,
               ObjectGroup &scene,
               LightSet &lights)
{
  Object::HitInfo hit = scene.Intersect(ray);
  if (hit.GetT() != Shape::NoIntersection) {
    Vector point = ray.GetPoint(hit.GetT());
    Vector normal = hit.GetNormal(point);

    // ...
    vypočítat osvětlení v bodě point s normálou normal
    Color color = vypočtená barva;

    return color;
  } else return Color(0, 0, 0);
}
```

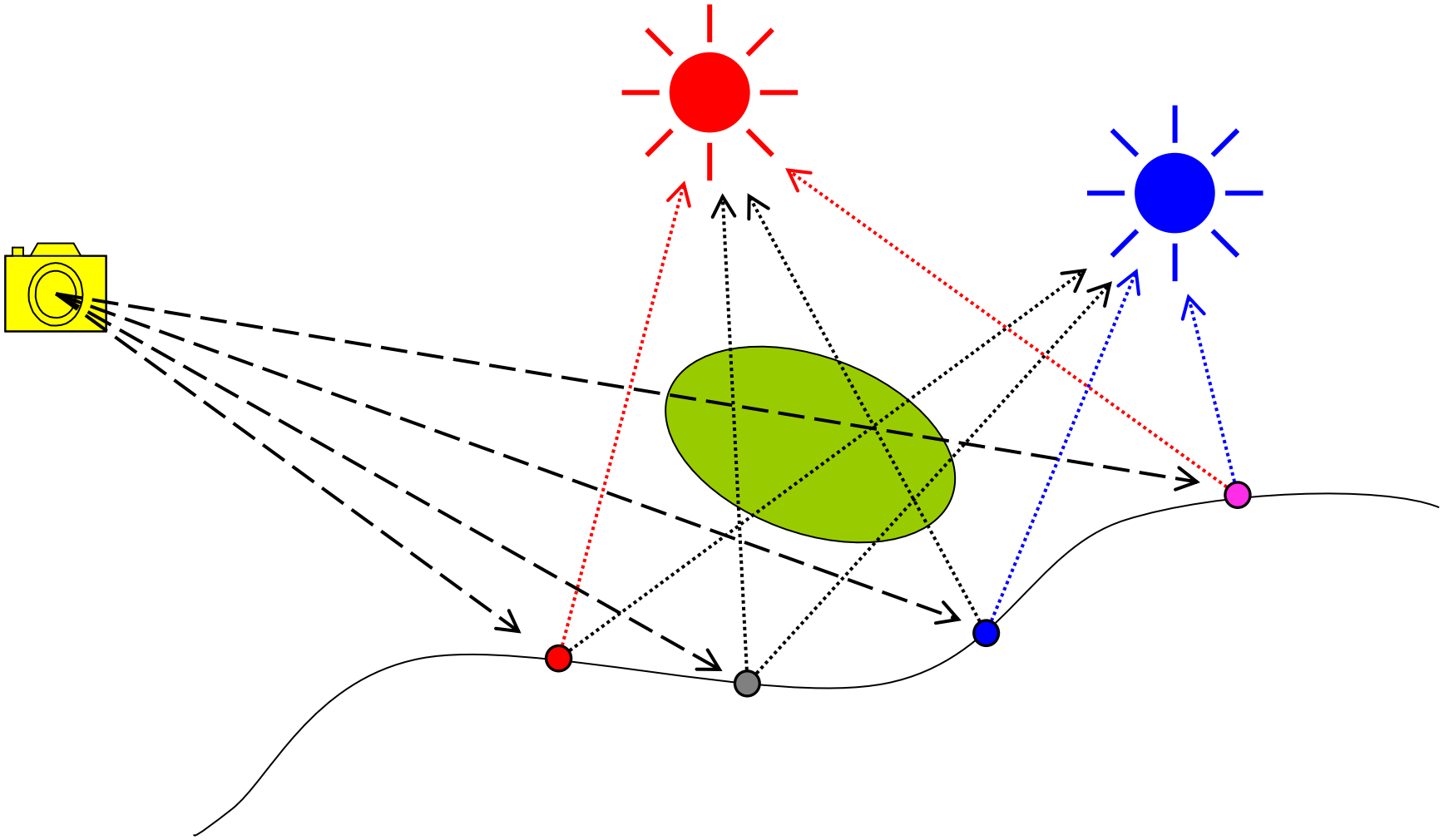
Sledování paprsku – pár pojmů

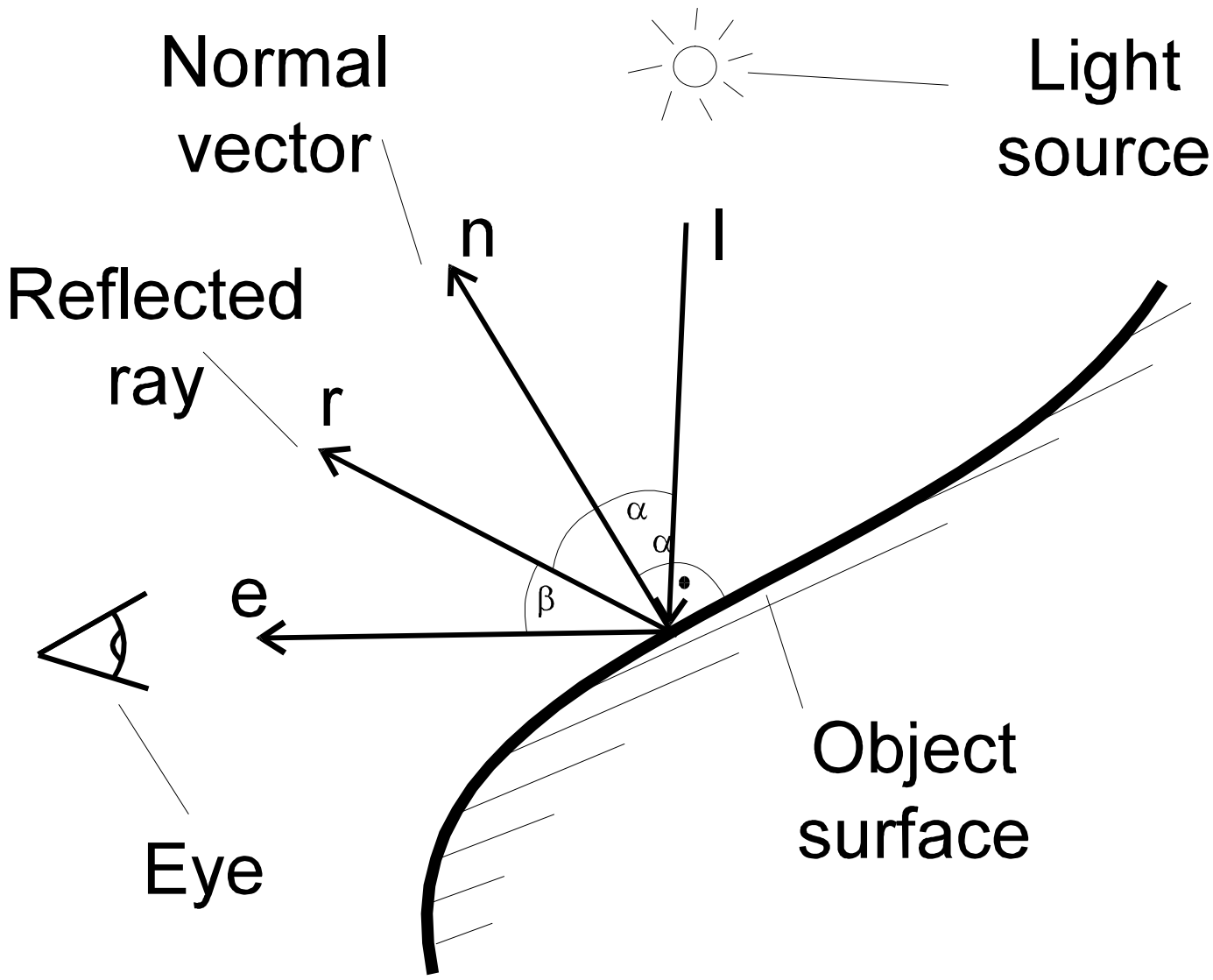
- Vrhání paprsku (Ray-Casting)

- Sledování paprsků – dopředné (ray tracing)
- Sledování paprsků – zpětné (Backward ray tracing, **ray tracing**)

- **Zpětné rekurzivní sledování paprsku**

Stínové paprsky




Normal
vector
Light
source
n
l
Reflected
ray
r
α
α
β
e
Eye
Object
surface

Osvětlení – Phongův model

$$\boldsymbol{I}_d = \boldsymbol{I}_l k_d \cos\alpha = \boldsymbol{I}_l k_d (\mathbf{l}\cdot\mathbf{n})$$

$$\boldsymbol{I}_s = \boldsymbol{I}_l k_s \cos^n \beta = \boldsymbol{I}_l k_s (\mathbf{e}\cdot\mathbf{r})^n$$

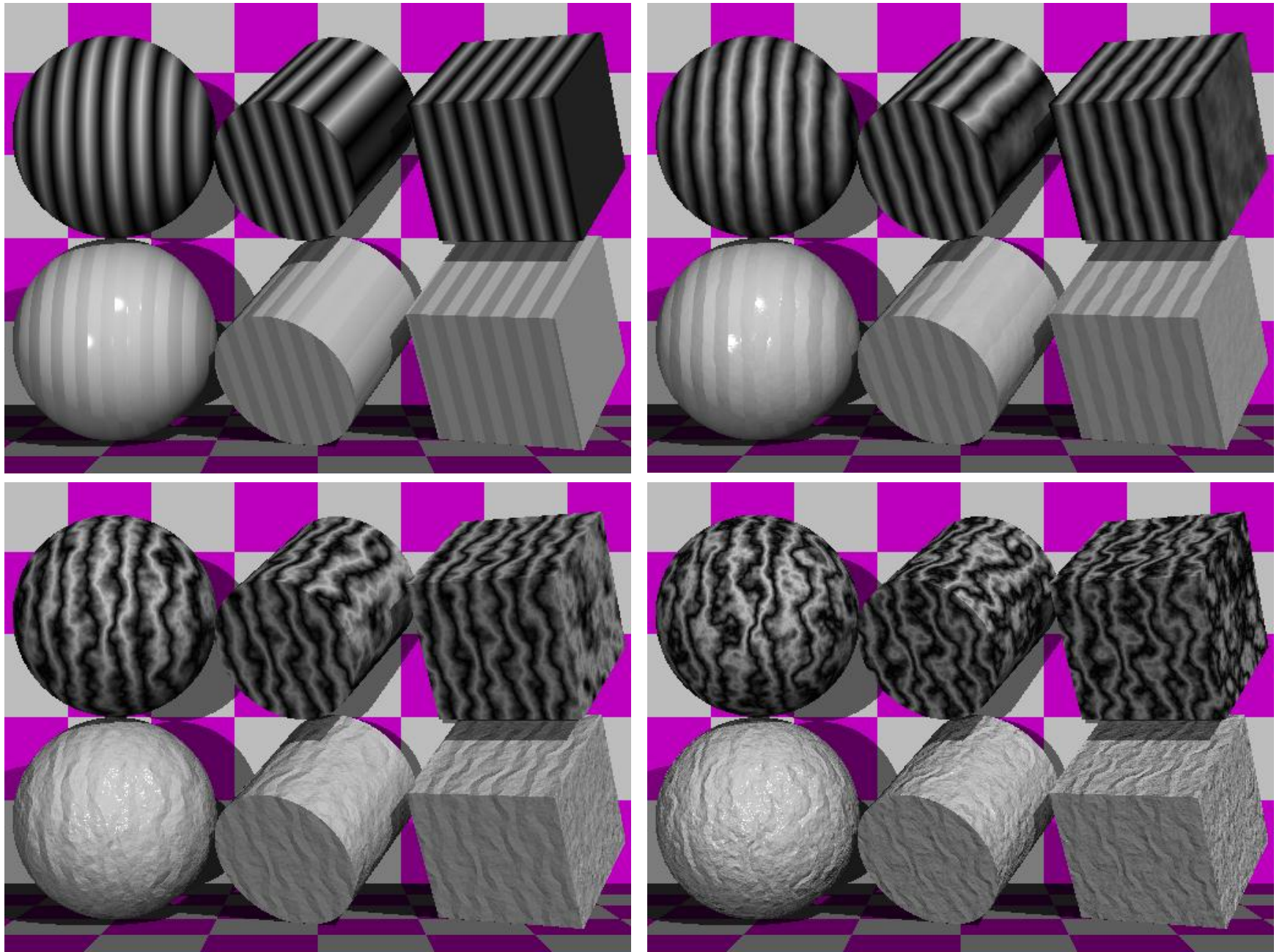
$$\mathbf{r} = \mathbf{l} - 2(\mathbf{l}\cdot\mathbf{n})\mathbf{n}$$

$$\boldsymbol{I} = \boldsymbol{I}_a + \boldsymbol{I}_d + \boldsymbol{I}_s$$

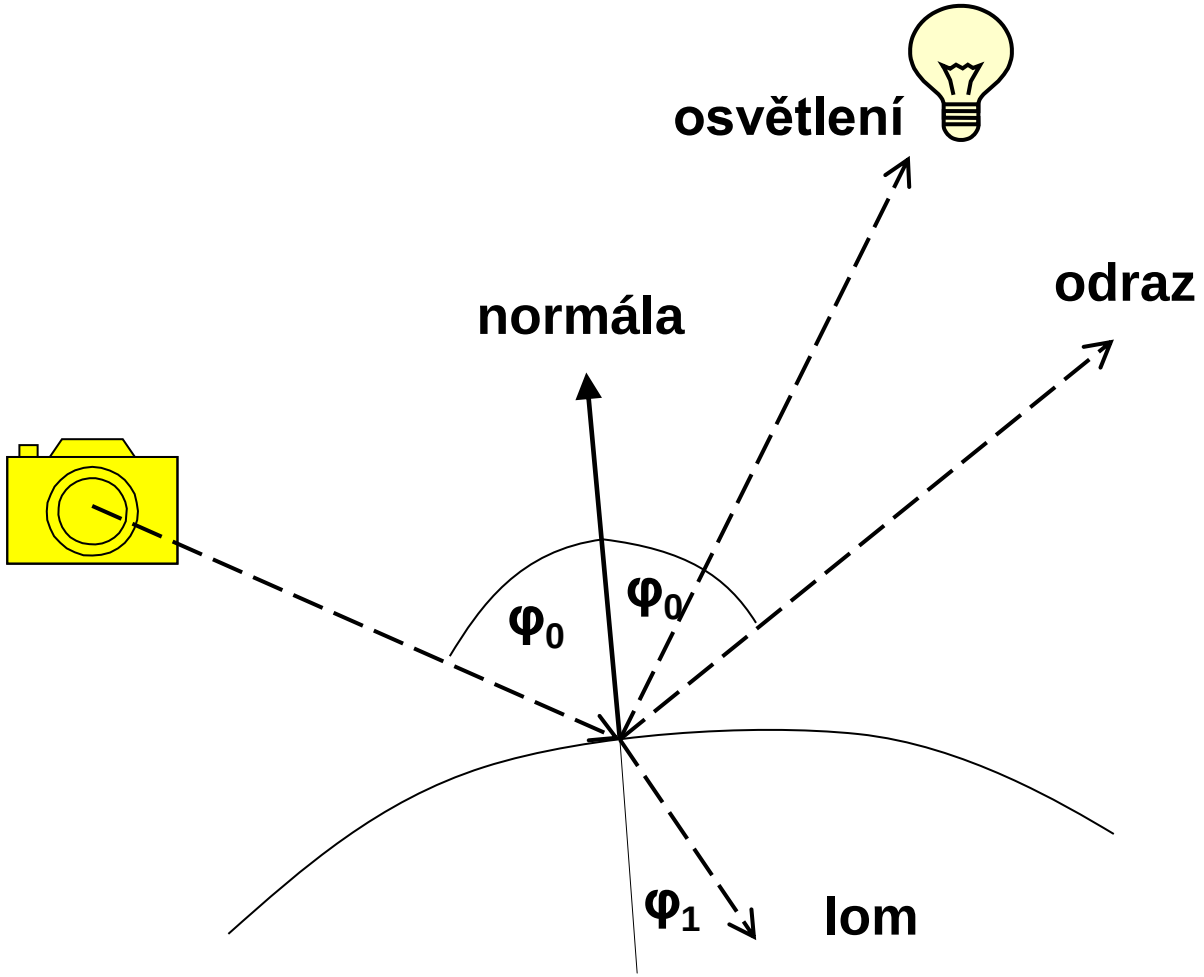
Phongův model osvětlení – ve funkci TraceRay

```
PhongInfo phong = hit.GetPhongInfo(point);
Color color = phong.GetAmbient();
for (LightSet::Iterator i = lights.begin(); i < lights.end(); i++) {
  Ray sray = (*i)->GetShadowRay(point);
  sray.ShiftStart();
  Object::HitInfo shit = scene.Intersect(sray);
  if (shit.GetT() > 1) {          // nejbližší průsečík je až za světlem
    // diffuse                    // tj. světlo je vidět
    Vector lv = sray.Direction();
    lv.Normalize();
    color.Accumulate(mult(phong.GetDiffuse(), (*i)->GetColor()),
                     fabs(lv*normal));
    // specular
    if (phong.GetShininess() != 0) {
      Vector rlv = 2*(lv*normal) * normal - lv;
      Vector vv = ray.Direction();  vv.Normalize();
      double specular = -(rlv*vv);
      if (specular > 0)
        color.Accumulate(mult(phong.GetSpecular(), (*i)->GetColor()),
                         pow(specular, phong.GetShininess()));
    }
  }
}
```

Phongův model s modifikací normály texturou

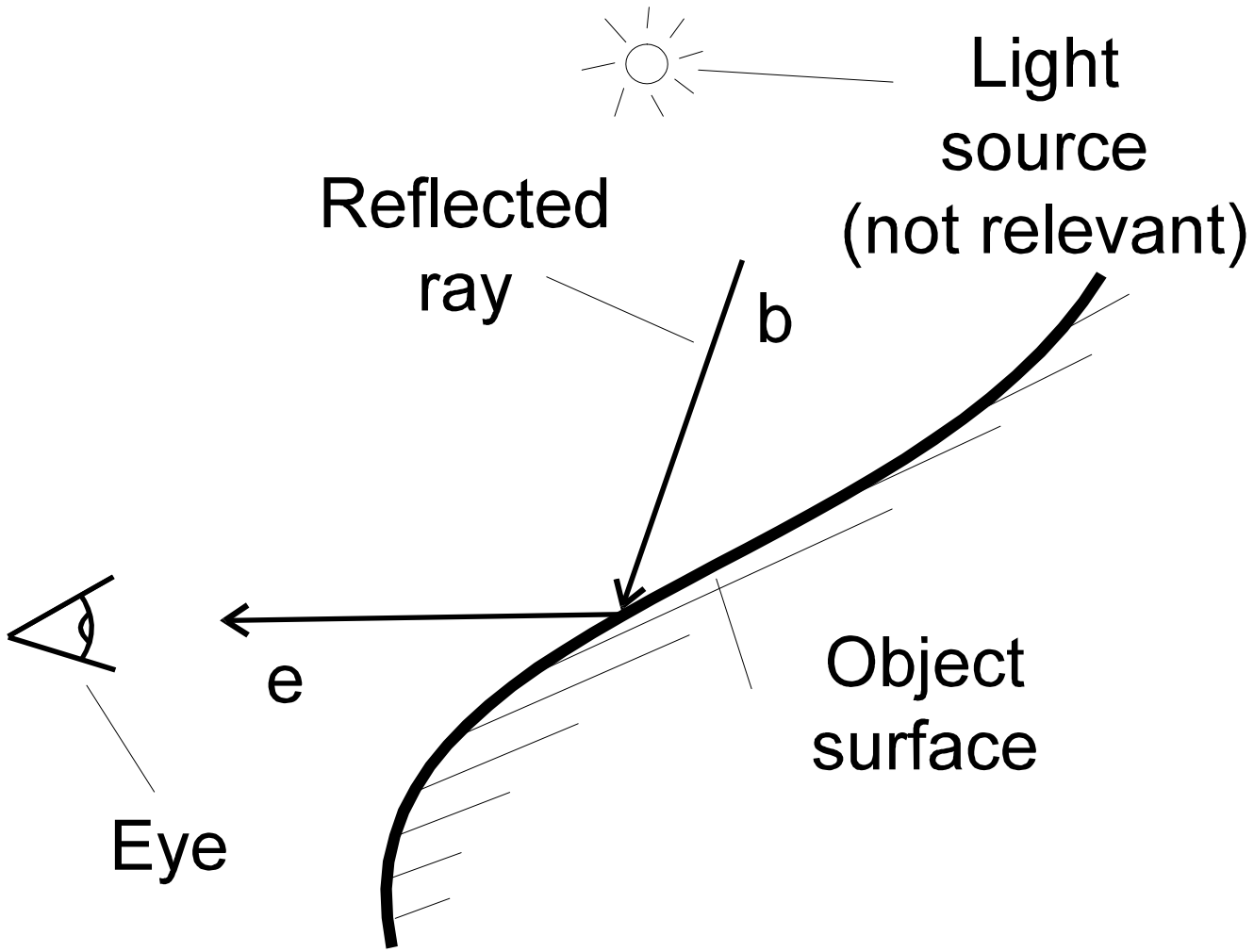


Sekundární paprsky

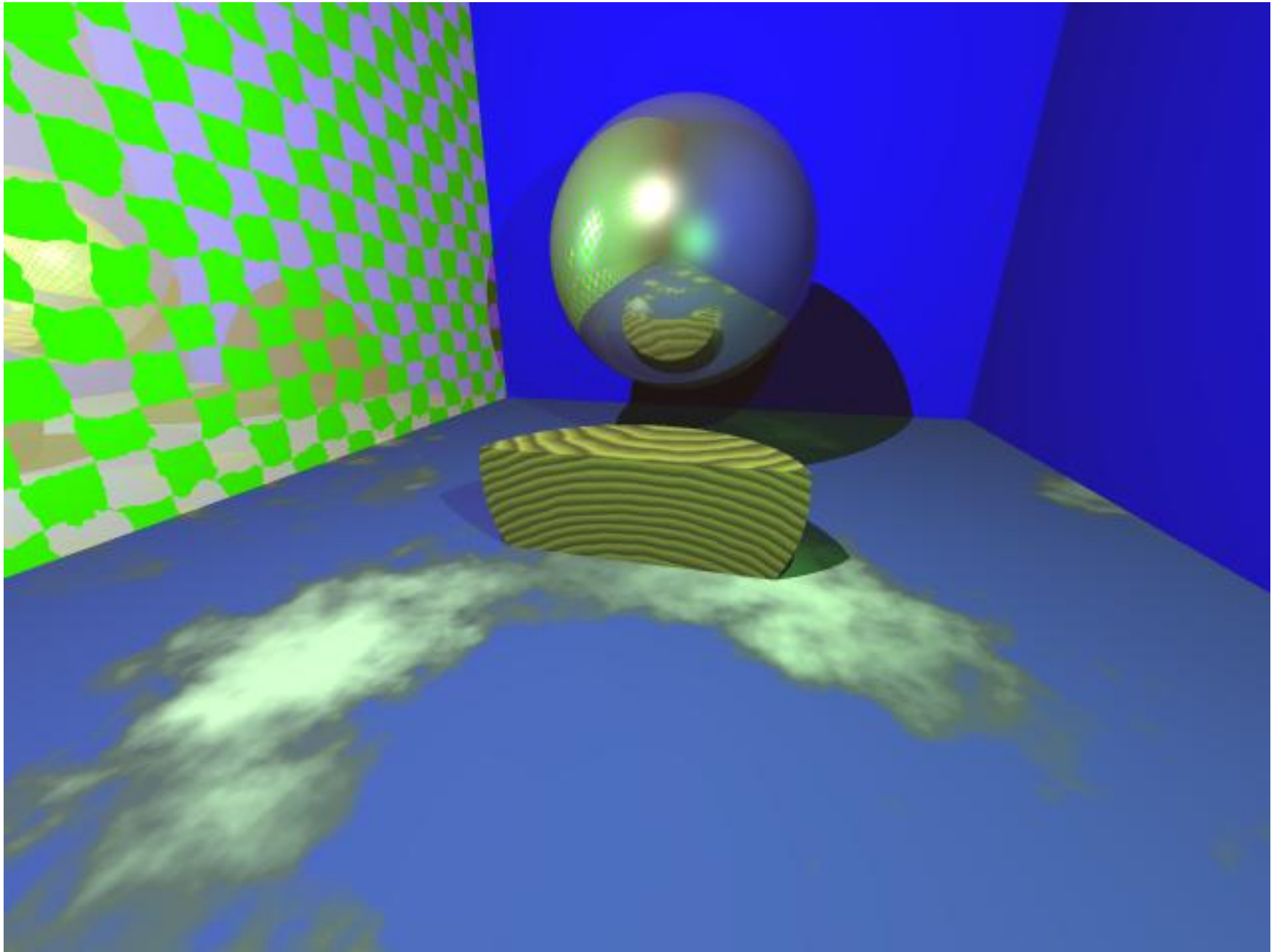


Odraz světla - Zrcadlo

- $I = k_r I_r$
- $r = e - 2(en)n$

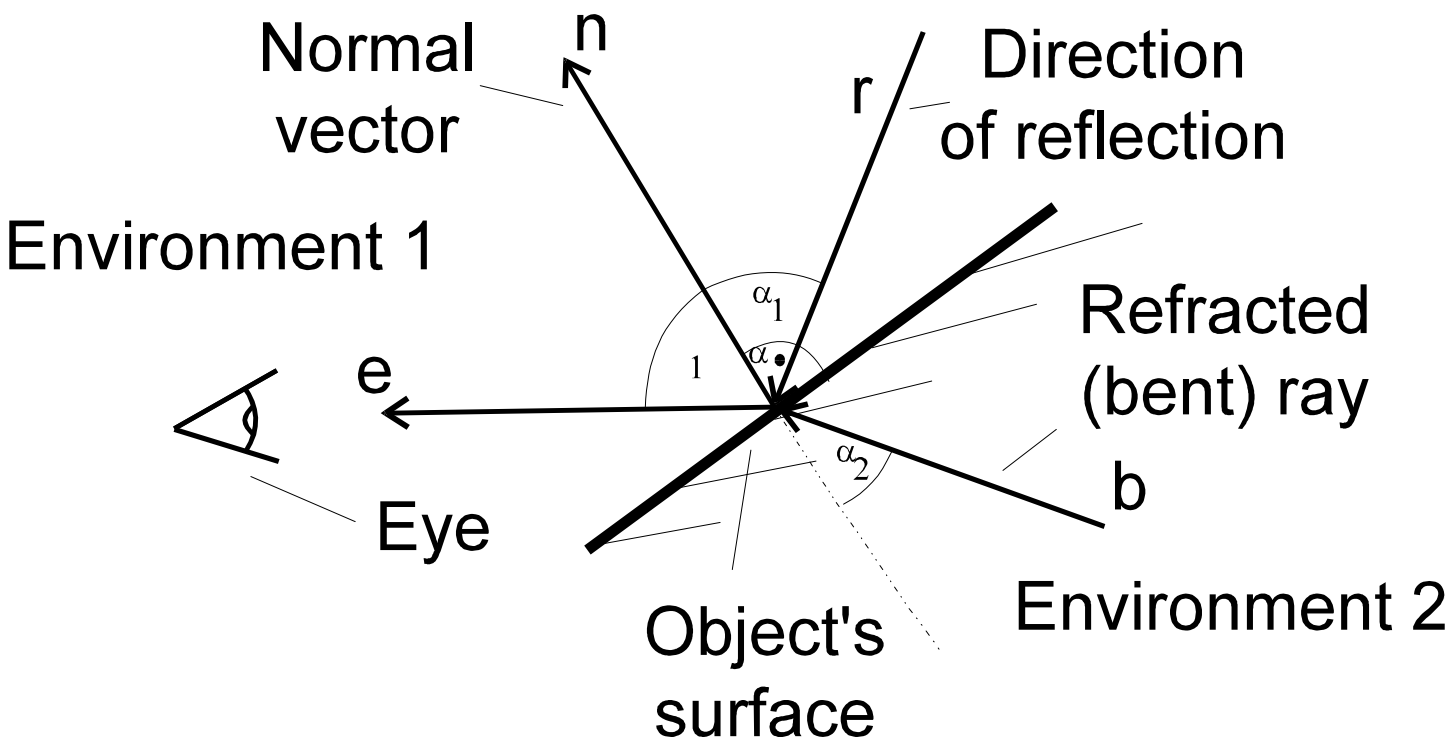


Zrcadlo - příklad

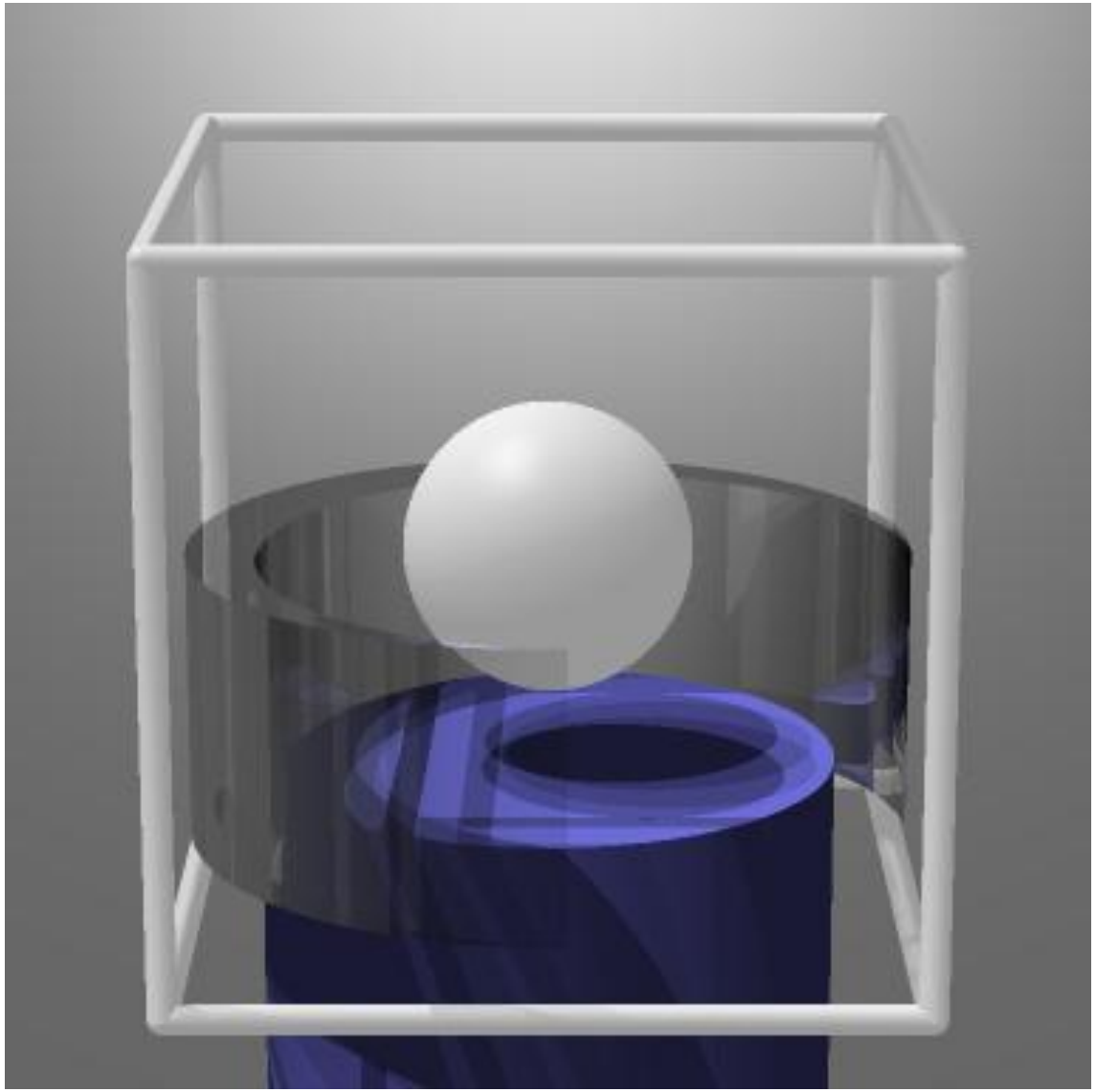


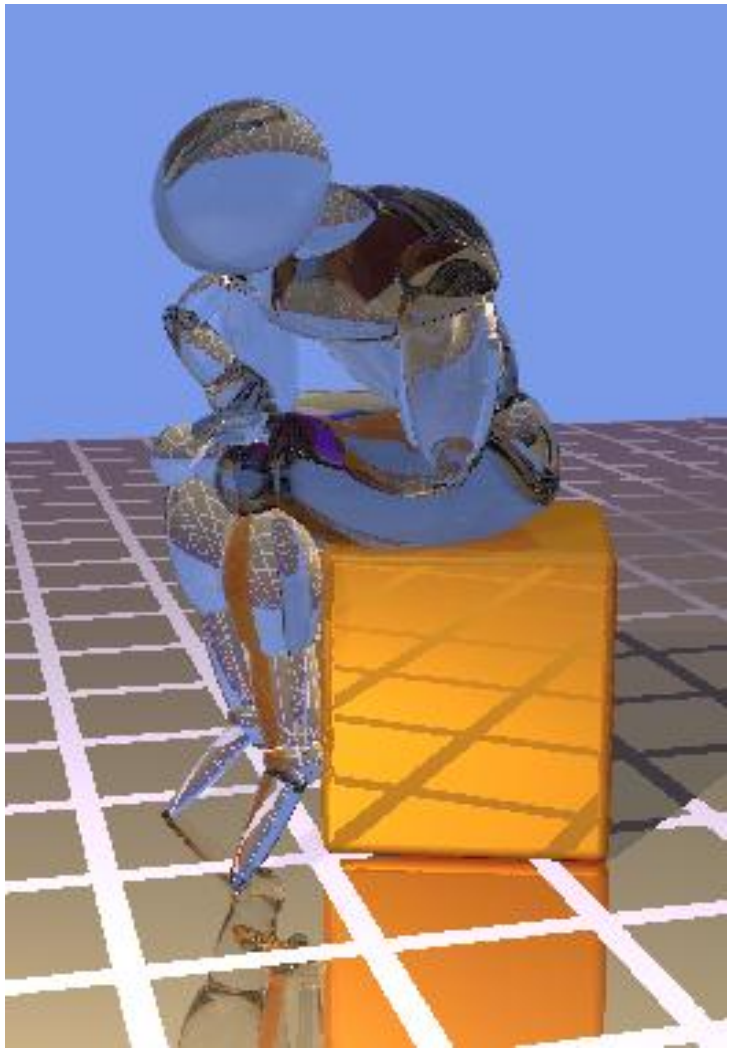
Lom světla - Sklo

- Úhel odrazu
  - $n_1 \sin \alpha_1 = n_2 \sin \alpha_2$
- Distribuce energie
  - $R = 0.1 + (\alpha / 90)^4$

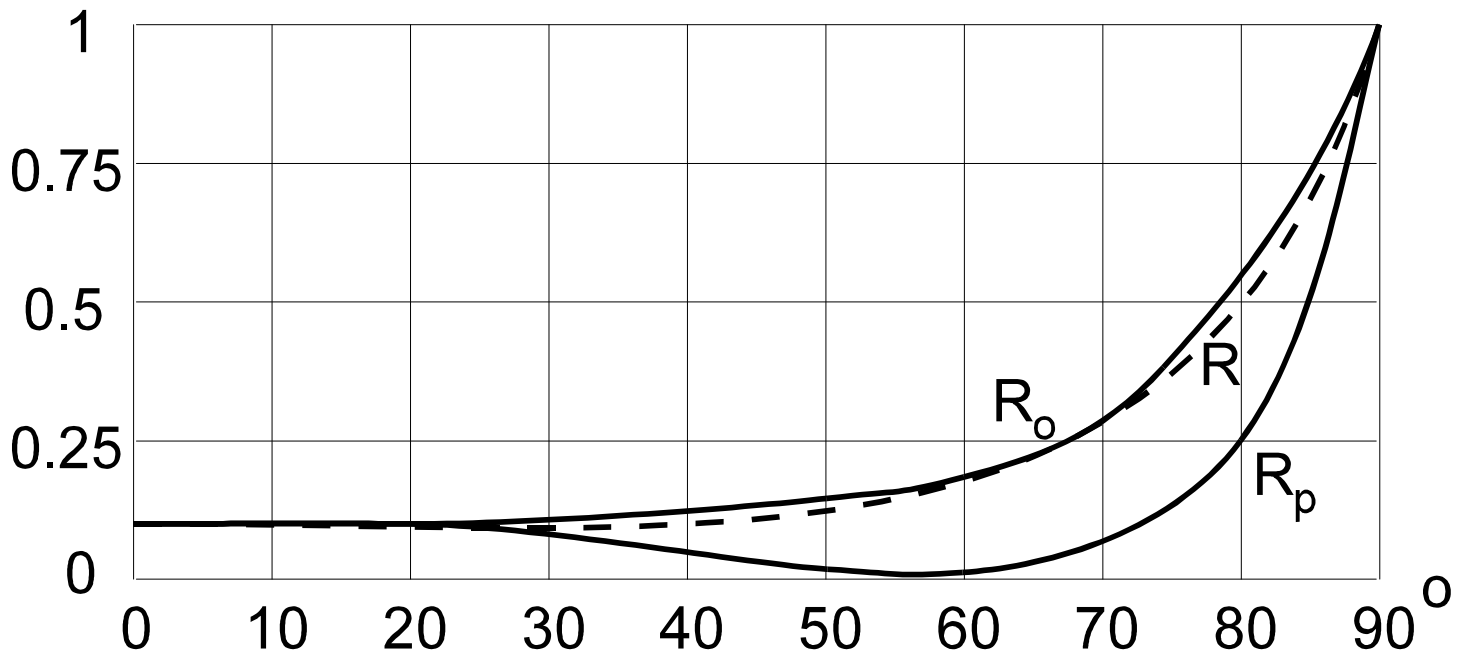


Sklo - příklad

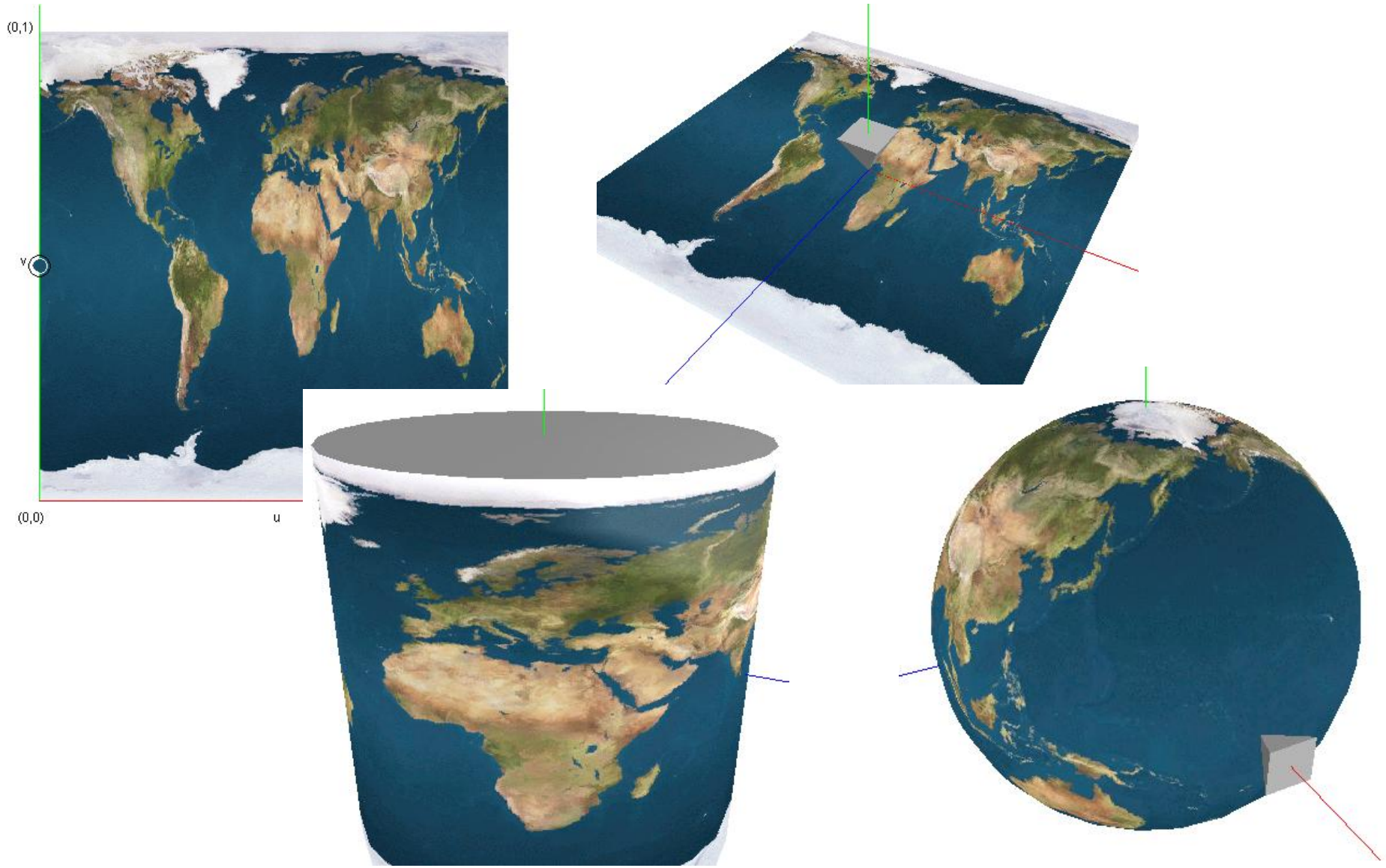




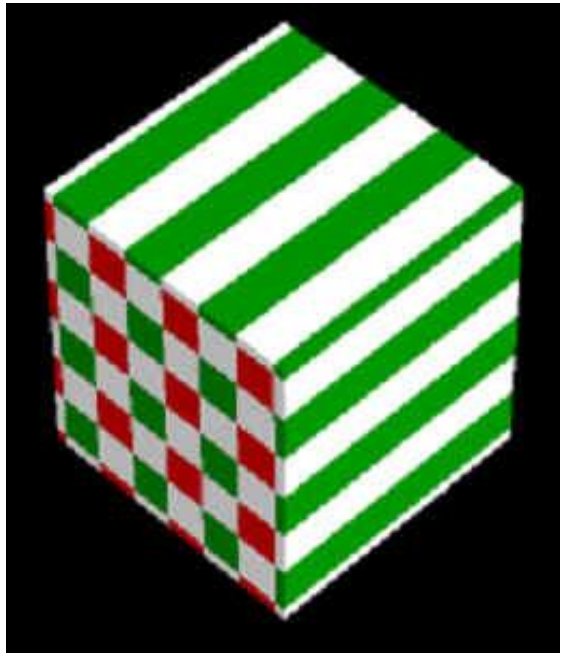
Odraz/Lom – distribuce energie

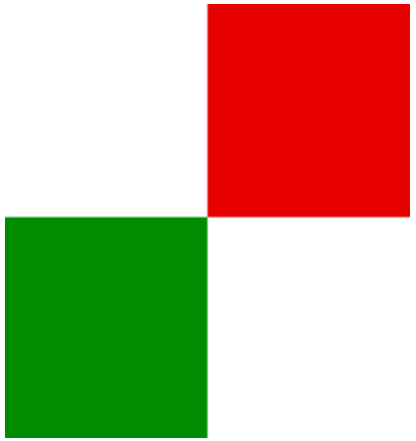


2DTexturování při sledování paprsku



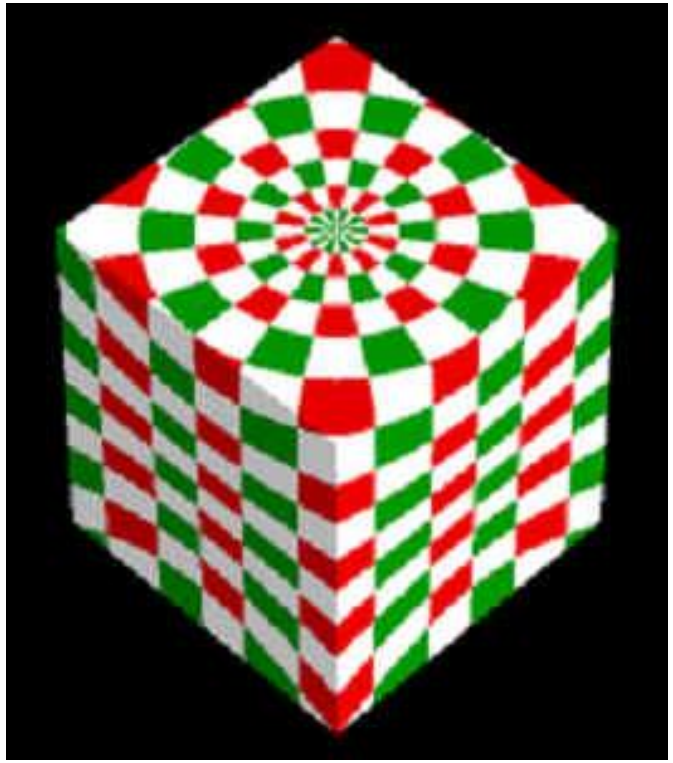
Planar texture mapping

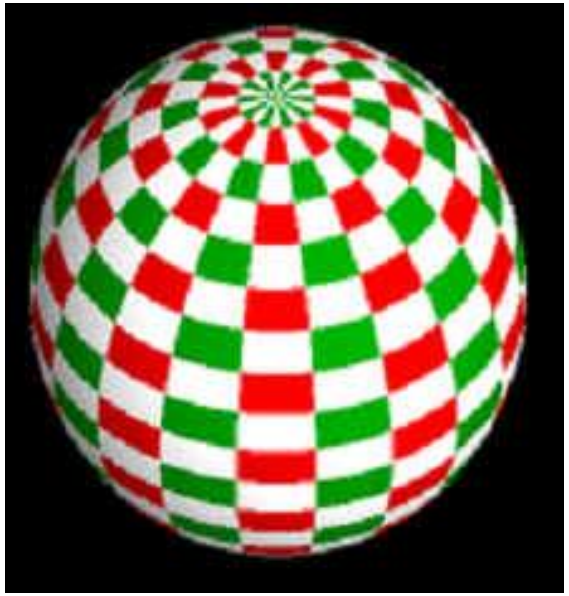


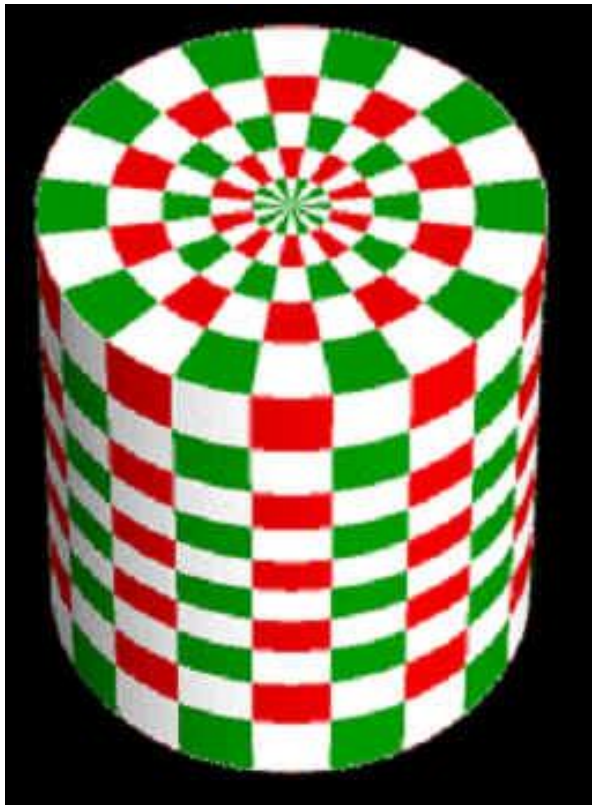


Cylindrical texture mapping

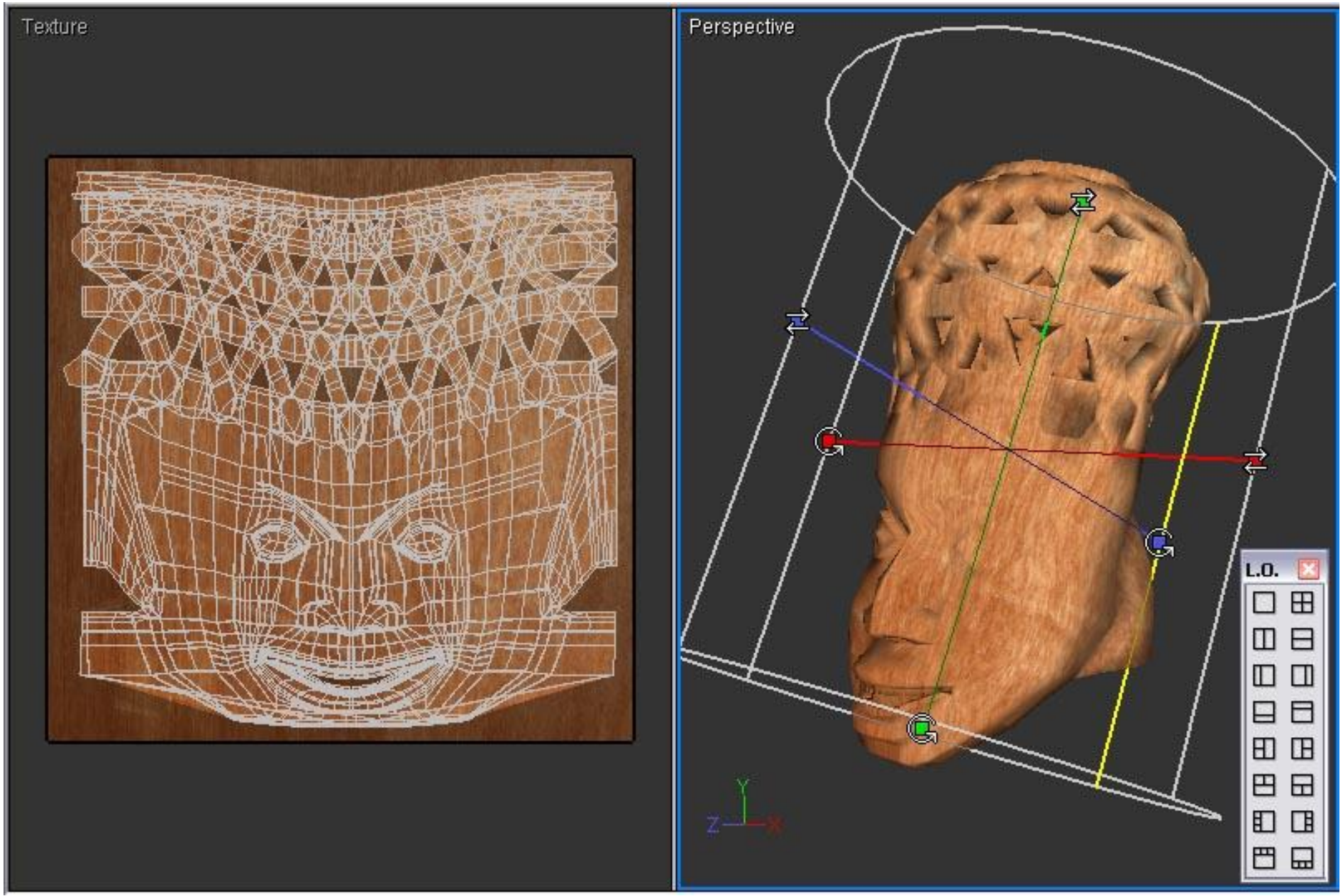
Spherical mapping



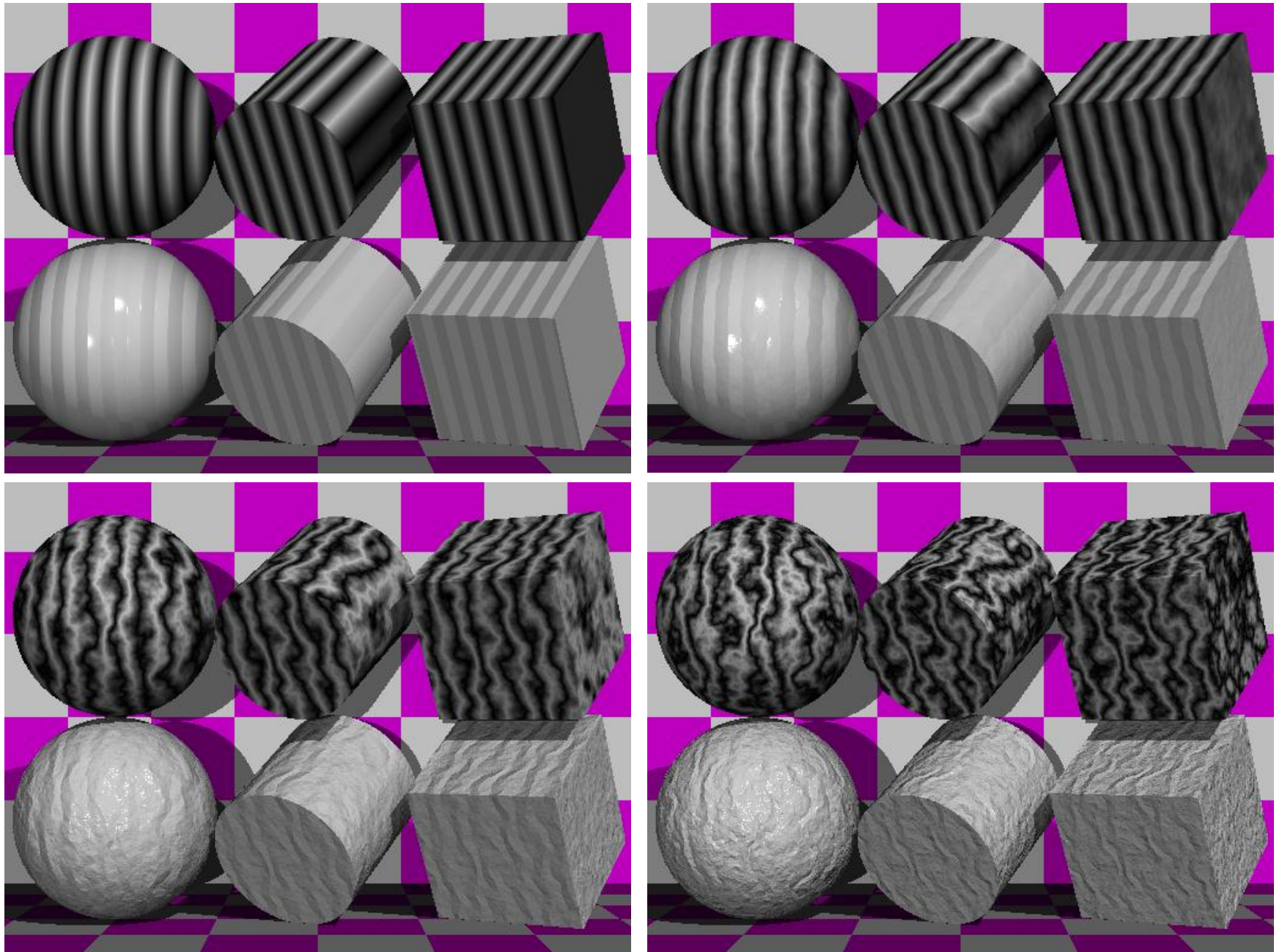




Mapování 2D textur – příklad: válcové mapování



Procedurální 3D texturování



Texturování při sledování paprsku

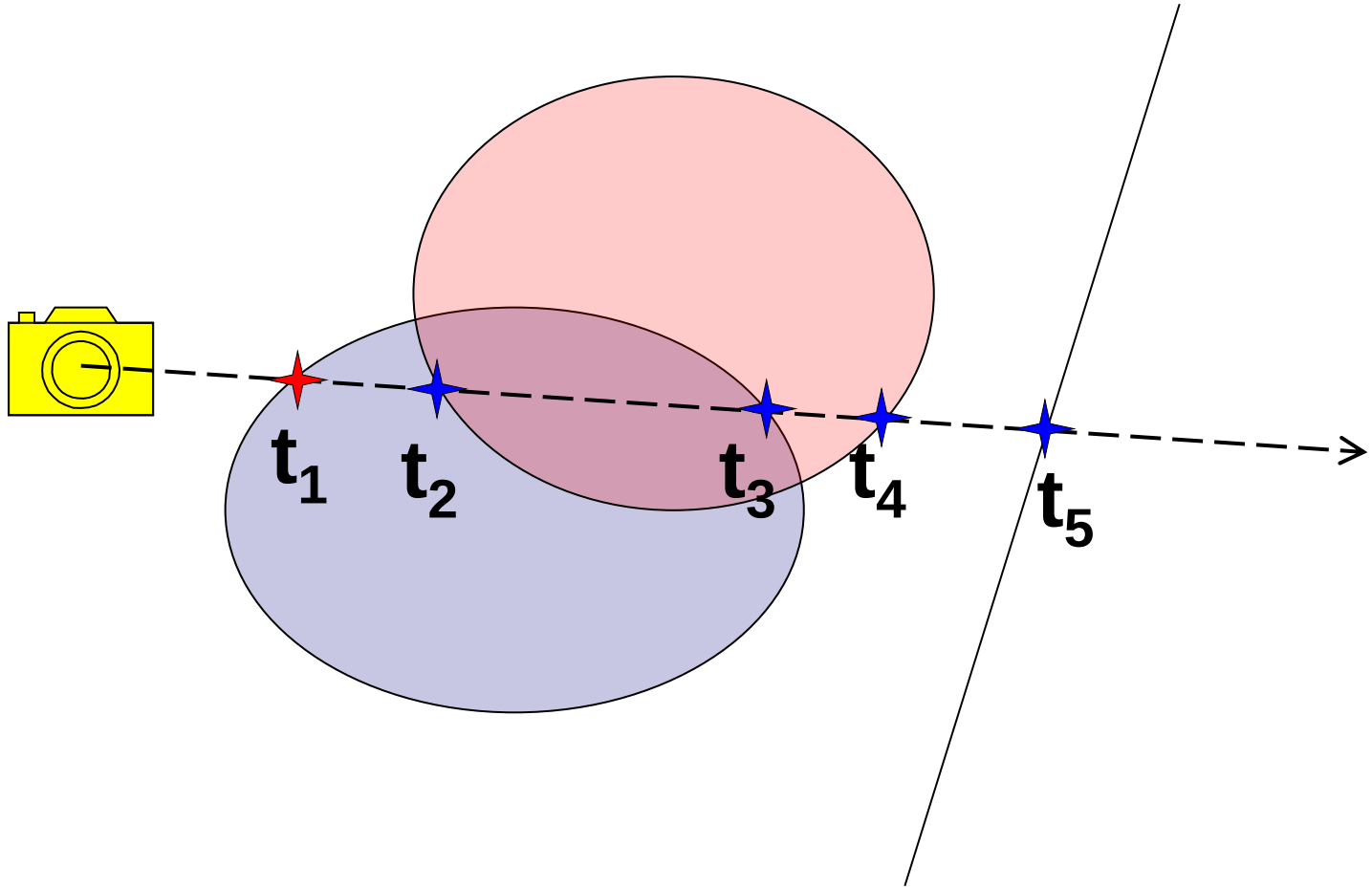
- Procedurální
- Z obrázků problematické
  - Texturovací souřadnice
  - Antialiasing
  - Kvalita

Texturování při sledování paprsku

- Pozn: antialiasing

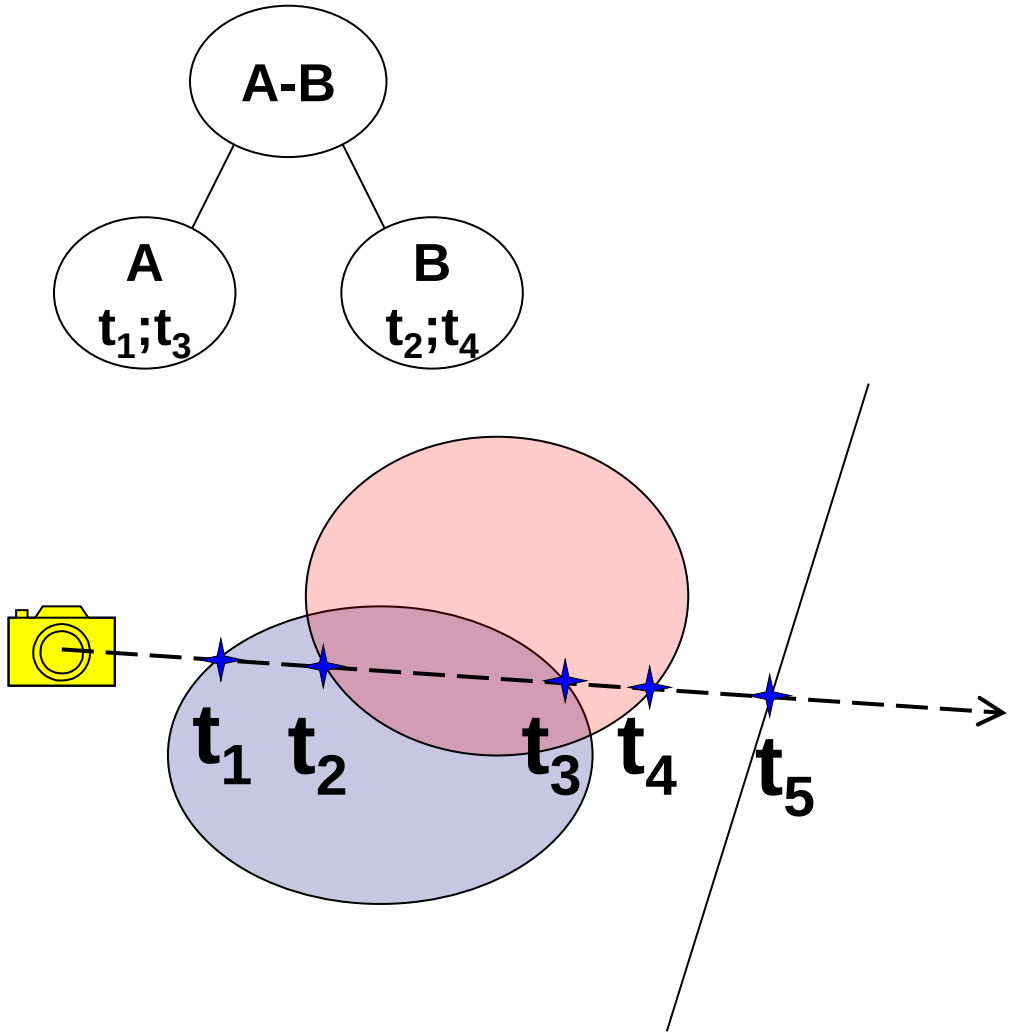
Nejbližší průsečík vs. CSG status tree

- „Události“ na paprsku



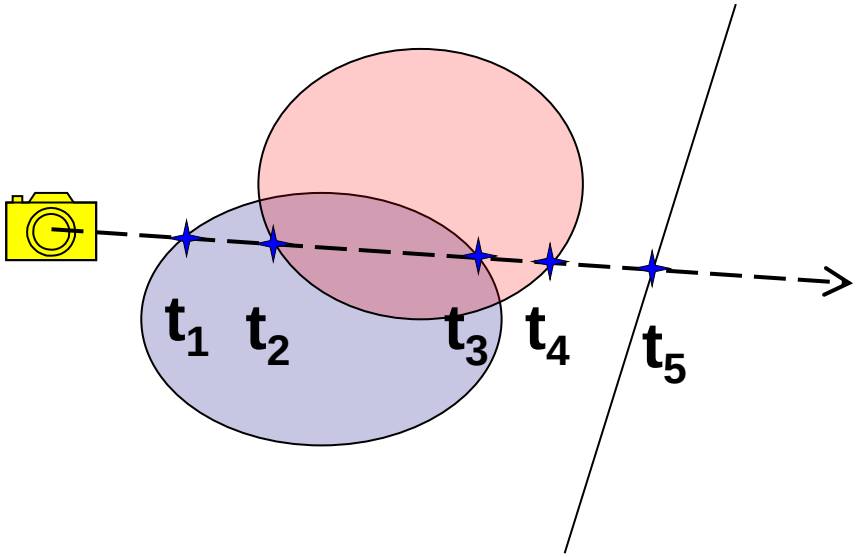
CSG a sledování paprsku

- „Události“ na paprsku
- Ohodnocení intervalů:
  - 0-t_1: { }
  - t_1-t_2: { A }
  - t_2-t_3: { A, B }
  - t_3-t_4: { B }
  - t_4-t_5: { }
  - t_5-?: { }
- Operace nad intervaly:
  - Průnik – první interval s { A,B }
  - A-B – první interval s { A }
  - B-A – první interval s { B }



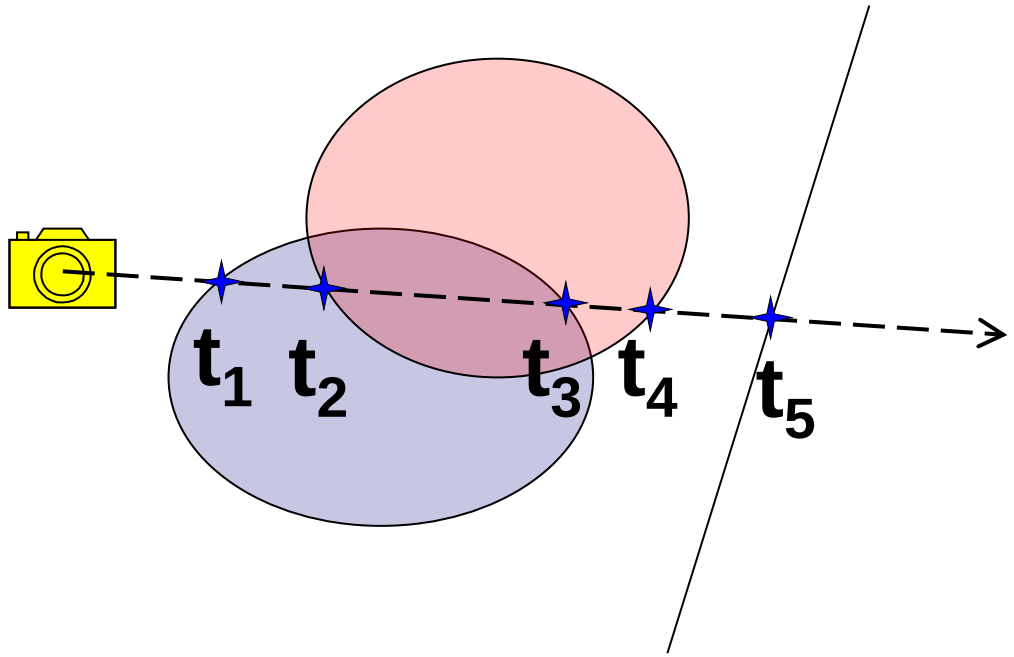
- CSG = Constructive Solid Geometry (konstruktivní geometrie těles)

CSG status tree



CSG – intervalové množiny

- Každé těleso přispěje intervalovou množinou
- CSG operace se provádějí nad intervaly
- Nejbližší průsečík odpovídá nejbližší „hraně“ množiny



Trochu programování – intervalové množiny (mírně brutální)

```
template <class PtrType> struct RangeEdge {
  double t;  PtrType *ptr;
  enum EdgeKind { ekNone, ekInto, ekOutfrom } kind;
  RangeEdge(double t, PtrType *ptr, EdgeKind kind) {
    this->t = t; this->ptr = ptr; this->kind = kind;
  }
};
template <class PtrType> class Range {
  double l;  double r;  PtrType *lp;  PtrType *rp;
public:
  Range(double a, double b, PtrType *ap = NULL, PtrType *bp = NULL) {
    if (a <= b) {
      l = a; lp = ap; r = b; rp = bp;
    } else {
      l = b; lp = bp; r = a; rp = ap;
    }
  }
  inline double Left() const { return l; }
  inline double Right() const { return r; }
  inline PtrType *LeftPtr() const { return lp; }
  inline PtrType *RightPtr() const { return rp; }
  inline void SetLeft(double value, PtrType *ptr) { l = value; lp = ptr; }
  inline void SetRight(double value, PtrType *ptr) { r = value; rp = ptr; }
};
```

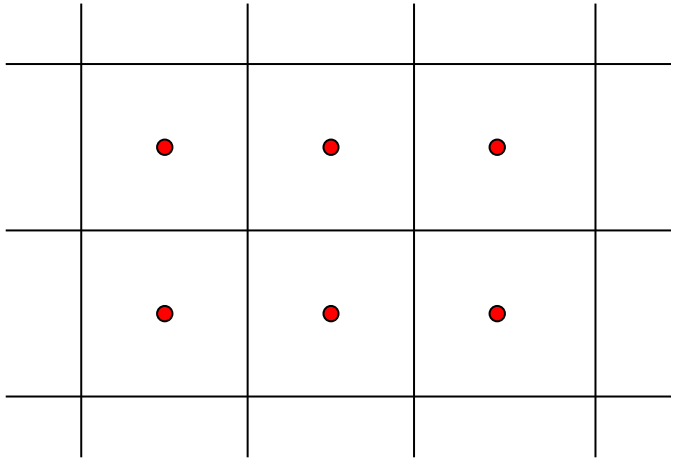
Trochu programování – intervalové množiny (mírně brutální)

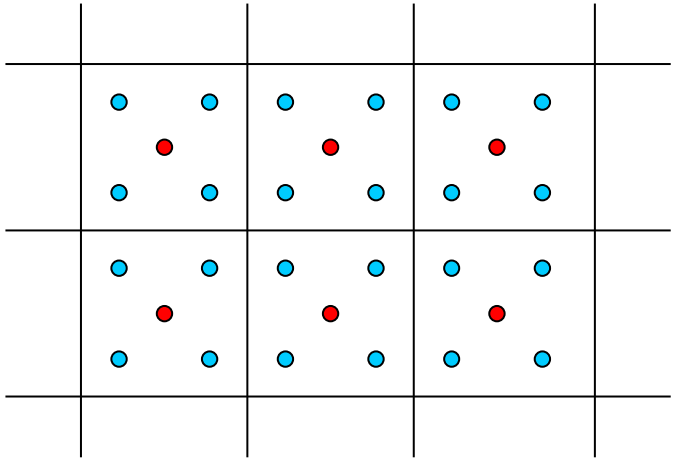
```
template <class T>
class Ranges {
  std::vector<Range<T> > data;
public:
  Ranges();
  Ranges(const Range<T> &r)
  bool Empty();
  struct RangeEdge<T> FirstEdgeGreater(double t = 0.0);
  void Add(const Range<T> &range);
  void ToStream(std::ostream &str);
  Ranges &operator += (Ranges &r);
  Ranges &operator *= (Ranges &r);
  void Inverse();
};
```

Antialiasing a Sledování paprsku

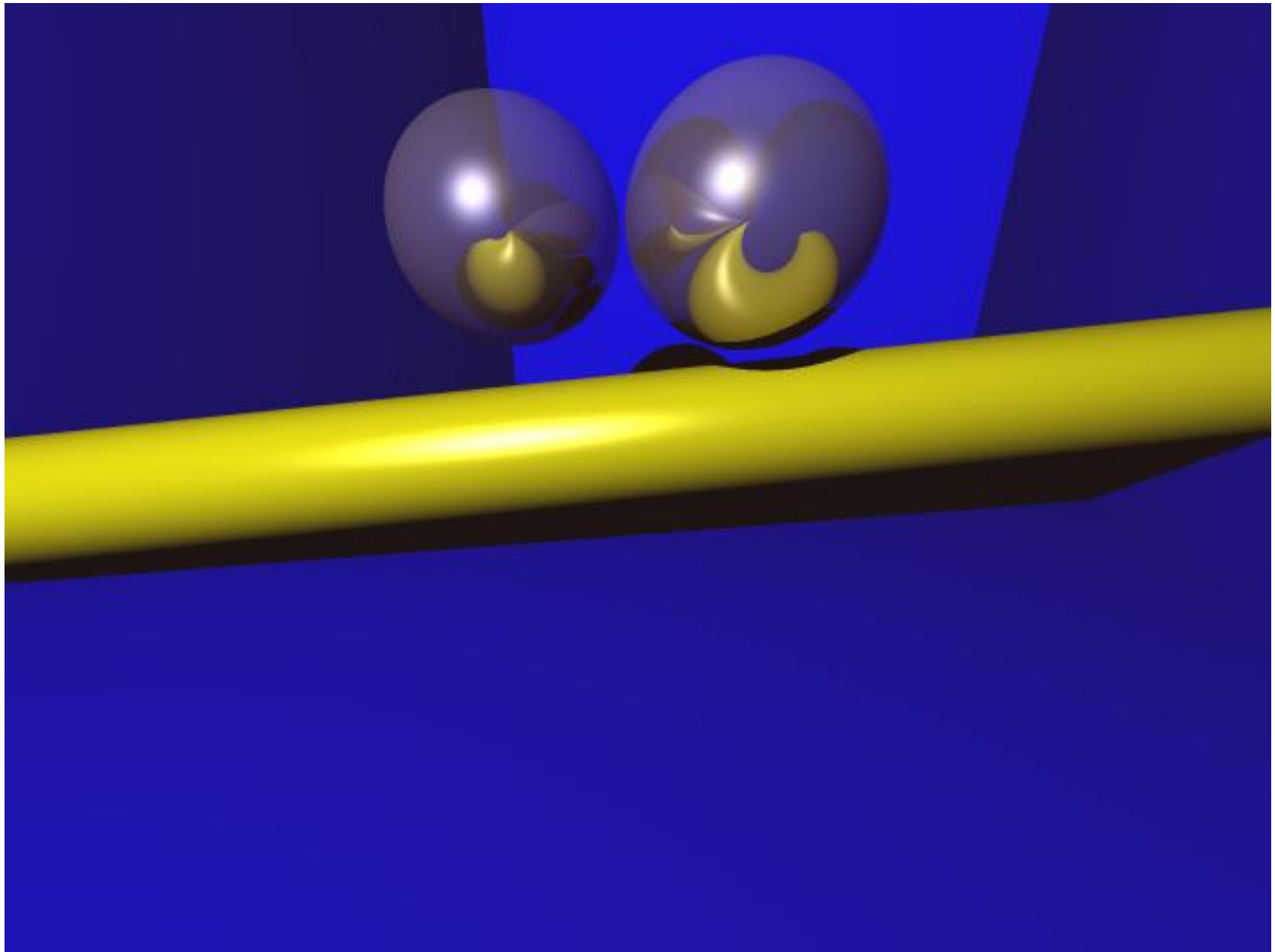
- Alias – „schody“
- Antialiasing – jak tomu předejít

- Supersampling

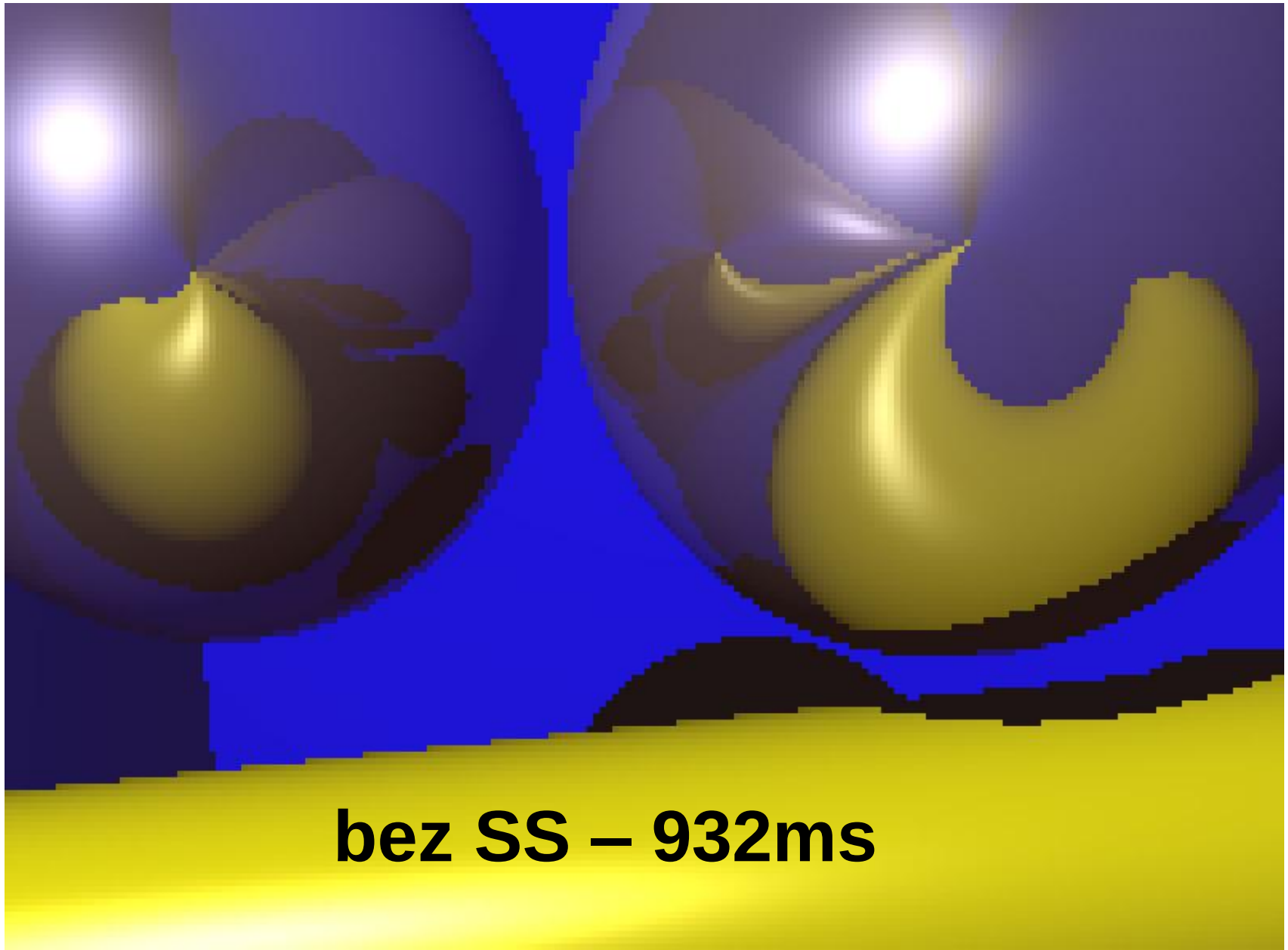




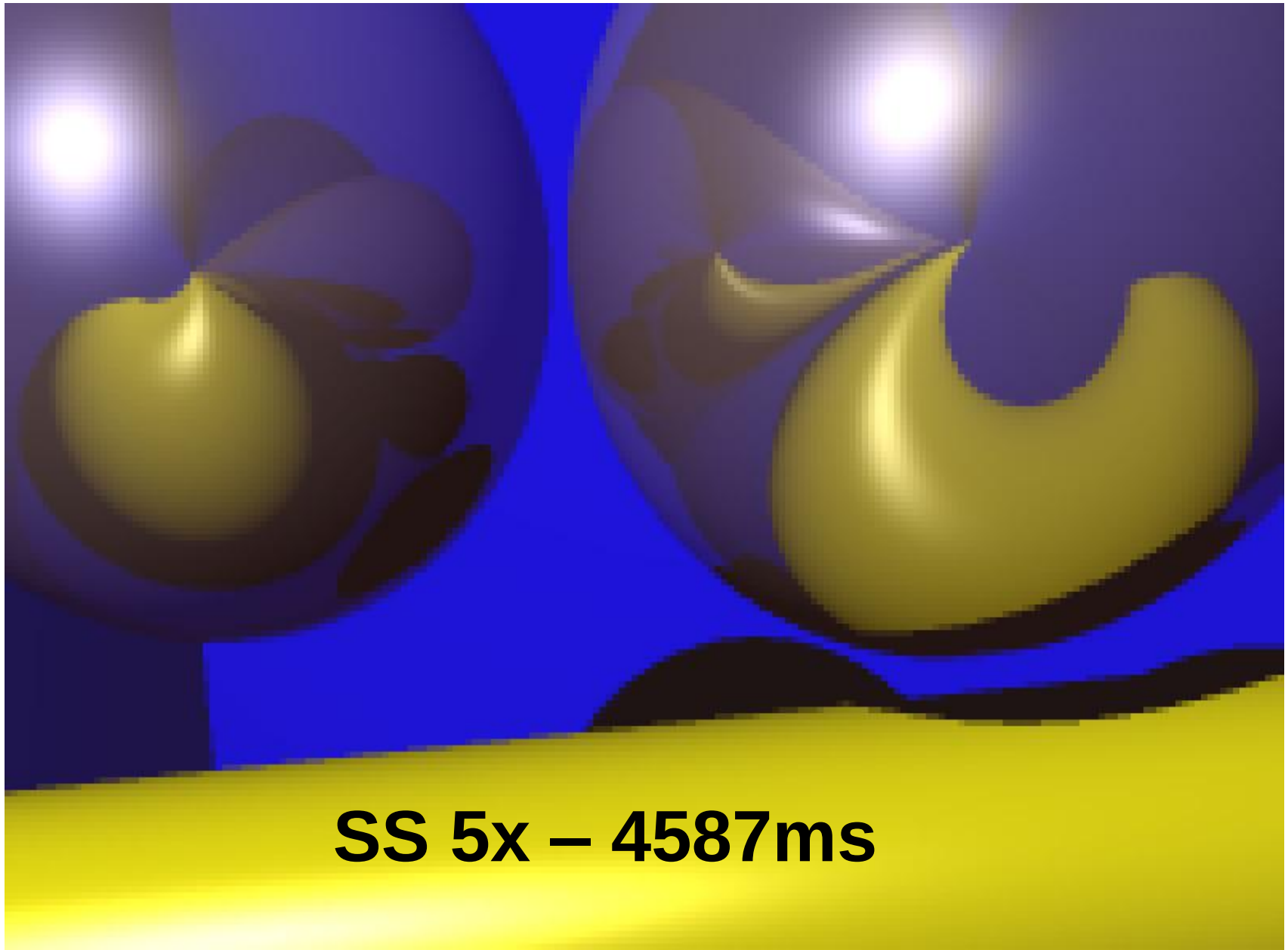
Supersampling – scéna



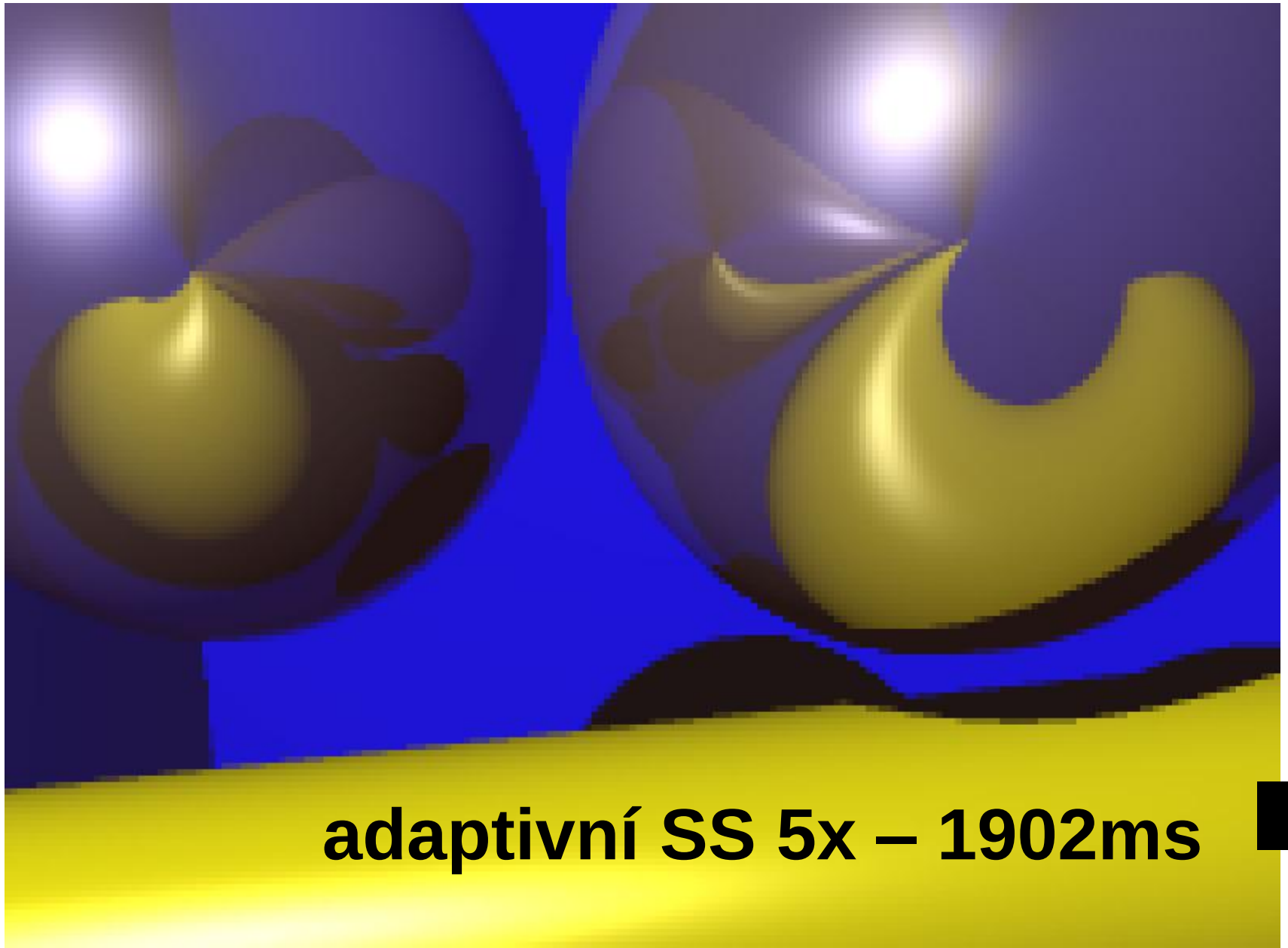
Supersampling – Adaptive Supersampling



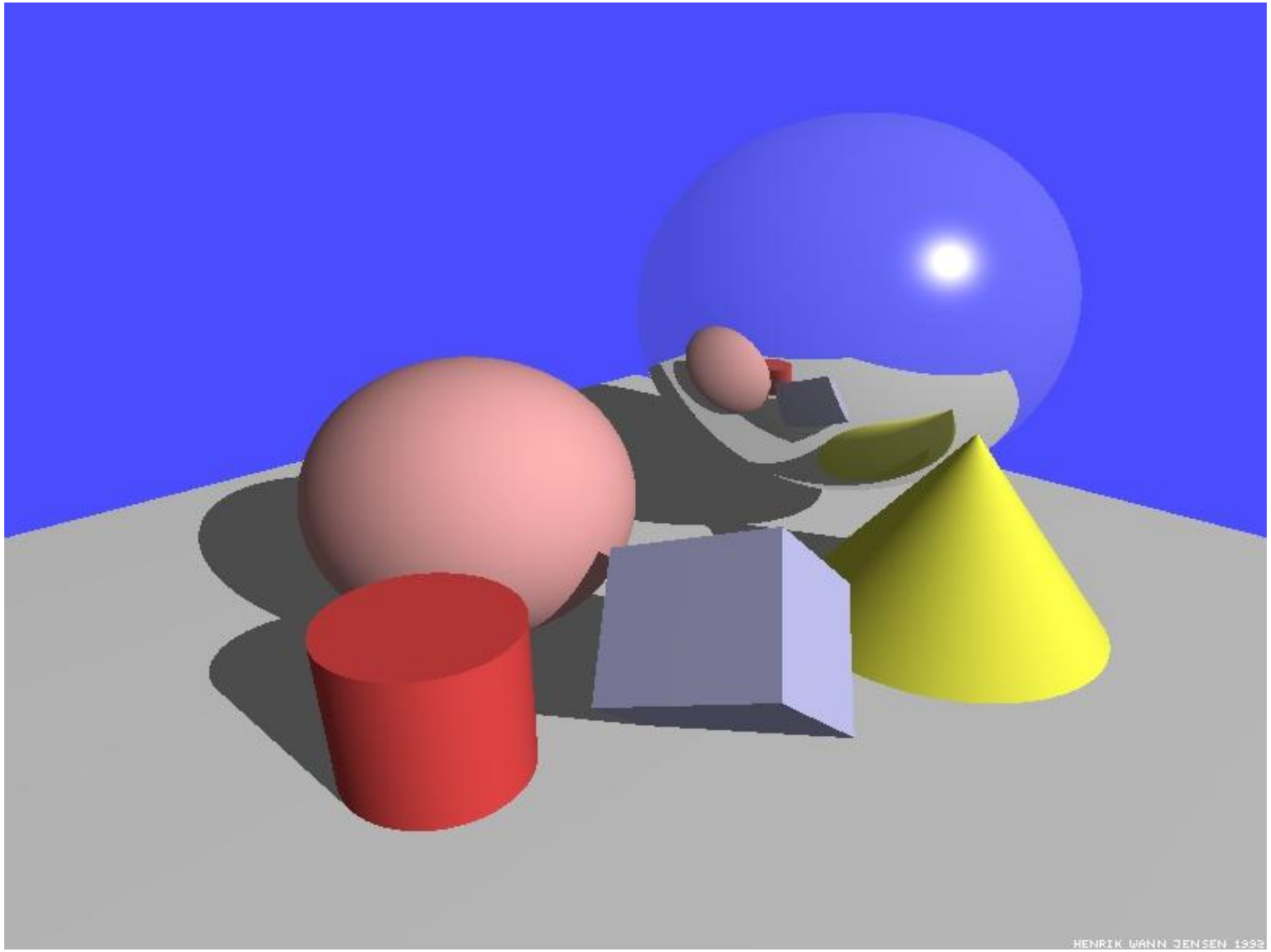
Supersampling – Adaptive Supersampling



Supersampling – Adaptive Supersampling



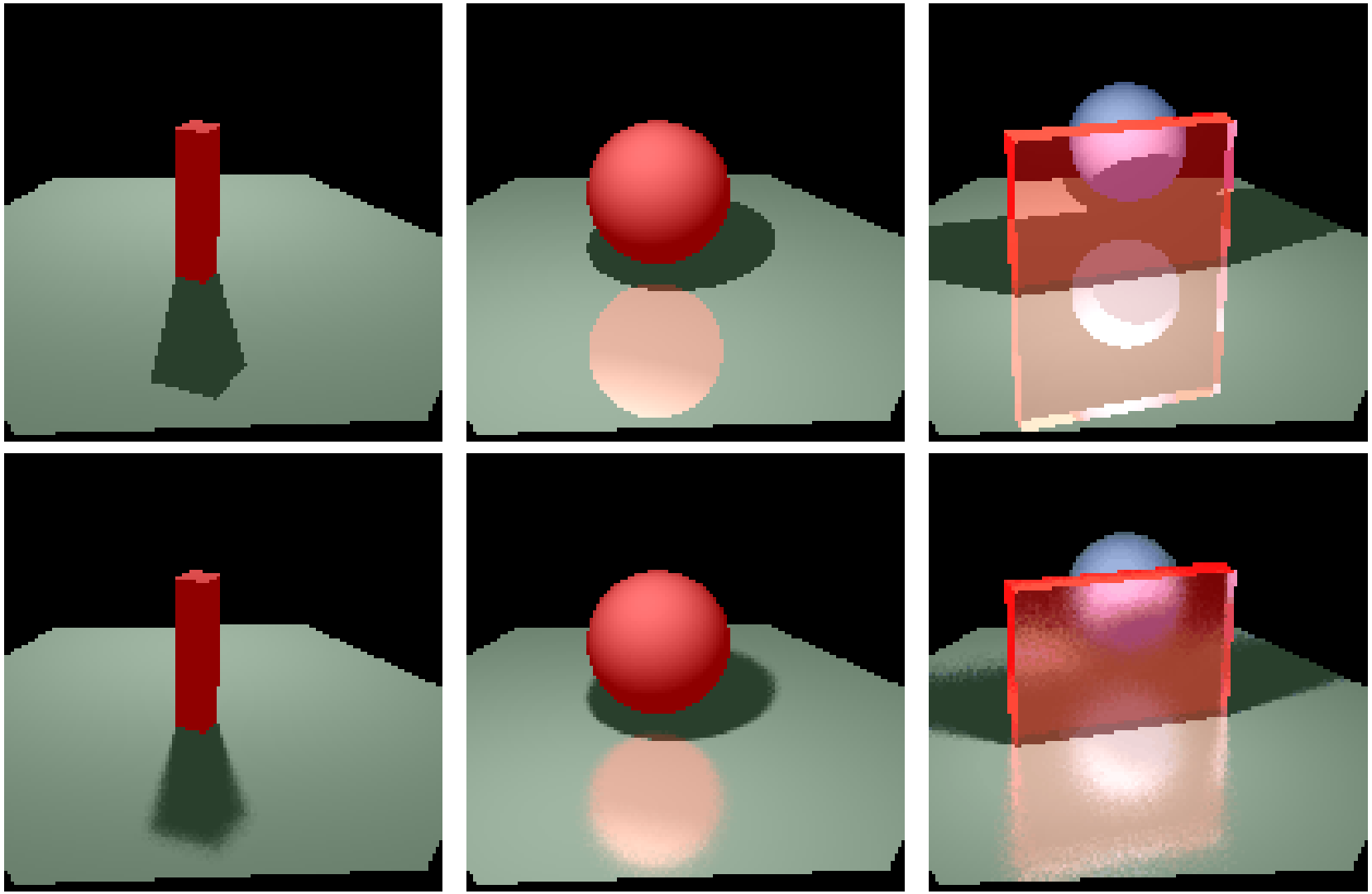
Sledování paprsku – obrázky, vlastnosti



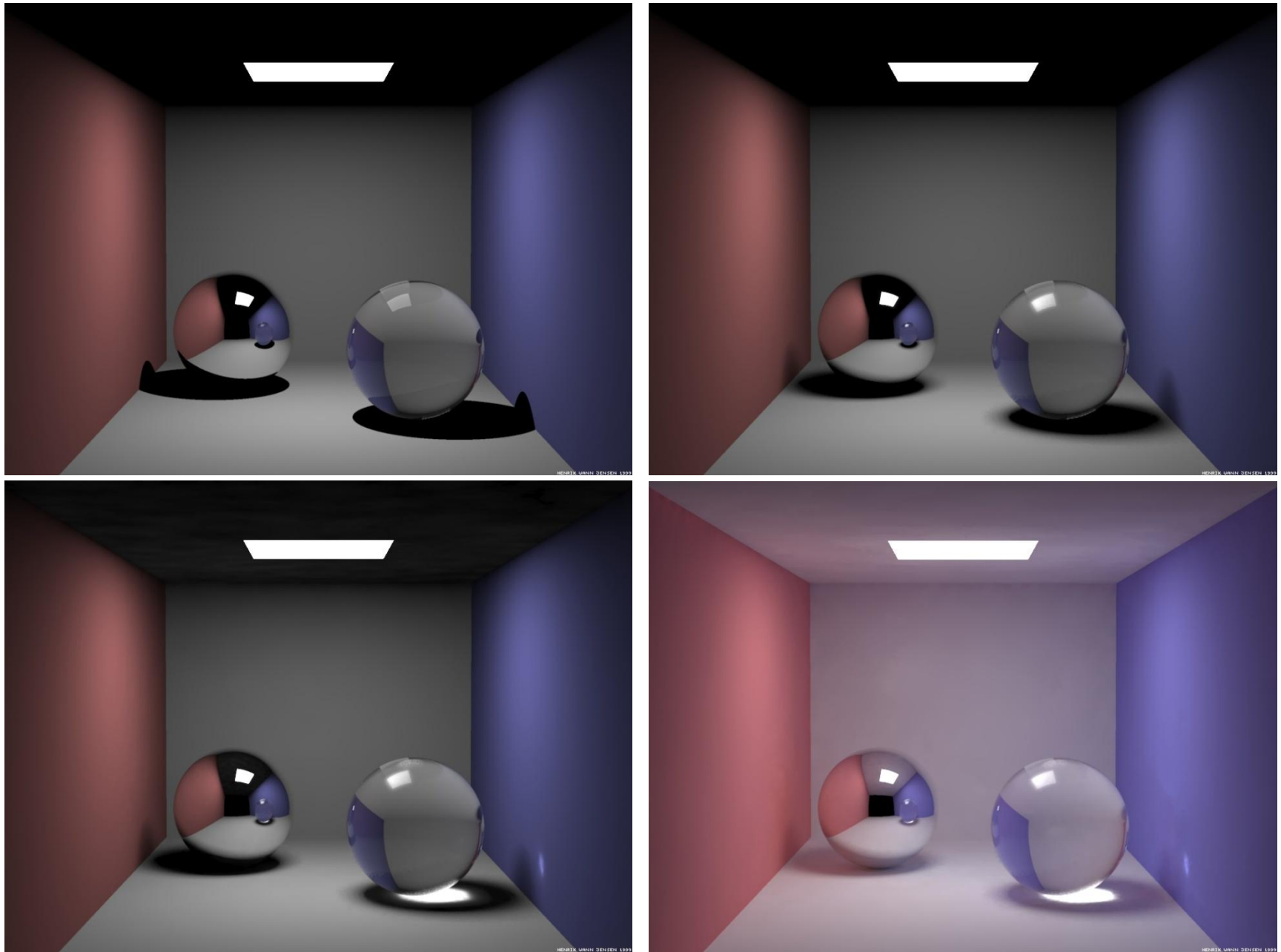
Sledování paprsku – obrázky, vlastnosti

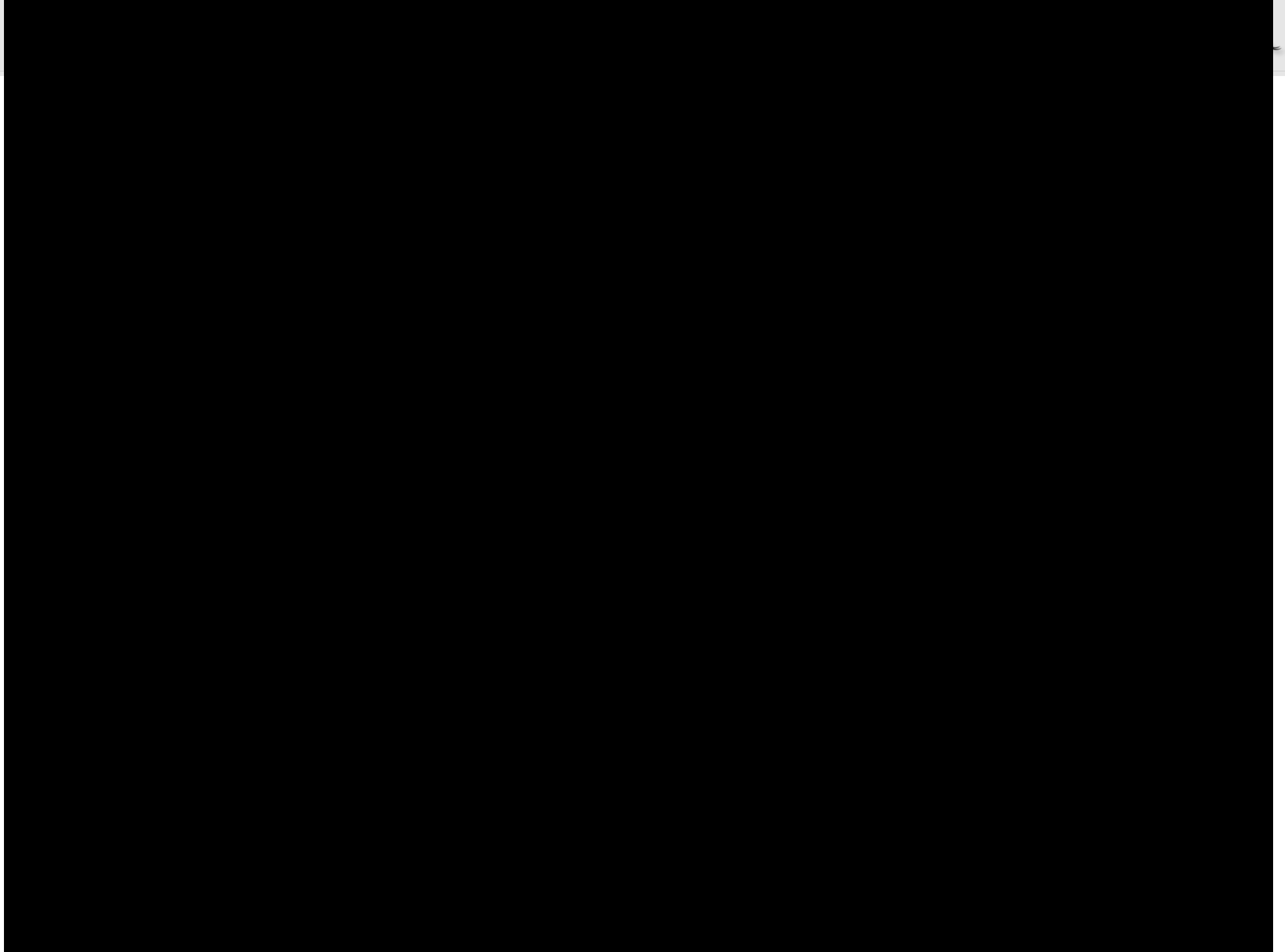
Sledování paprsku – obrázky, vlastnosti

Ray tracing – obrázky, vlastnosti



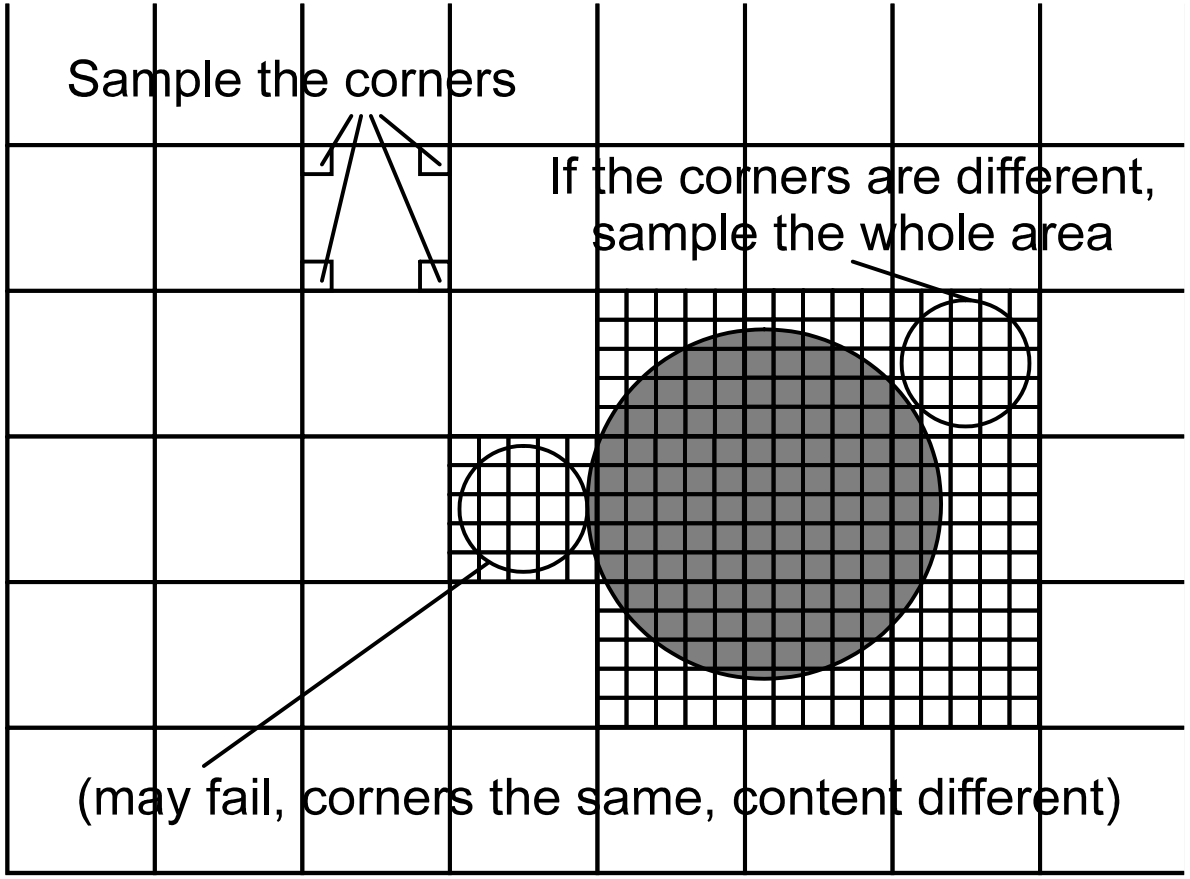
Ray tracing – obrázky, vlastnosti



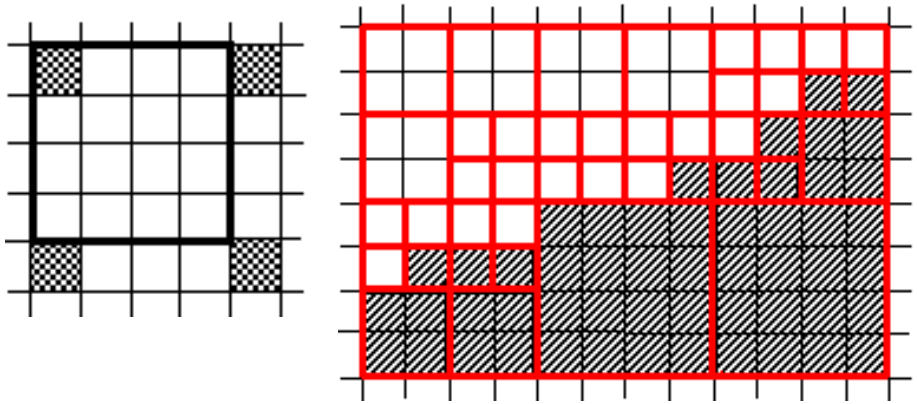


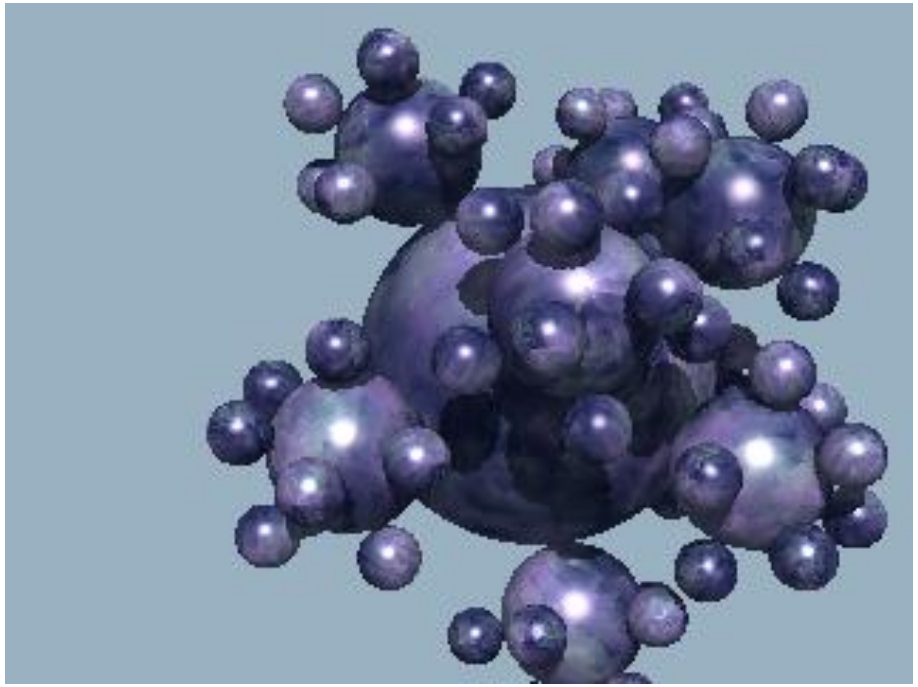
Adaptive subsampling

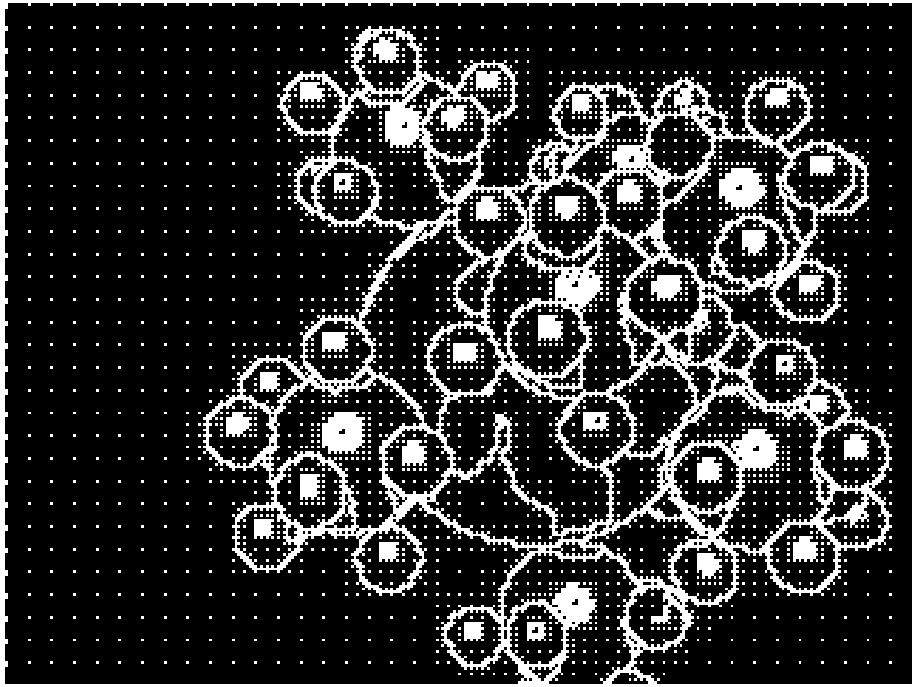
- Speedup ~3x (5x5)



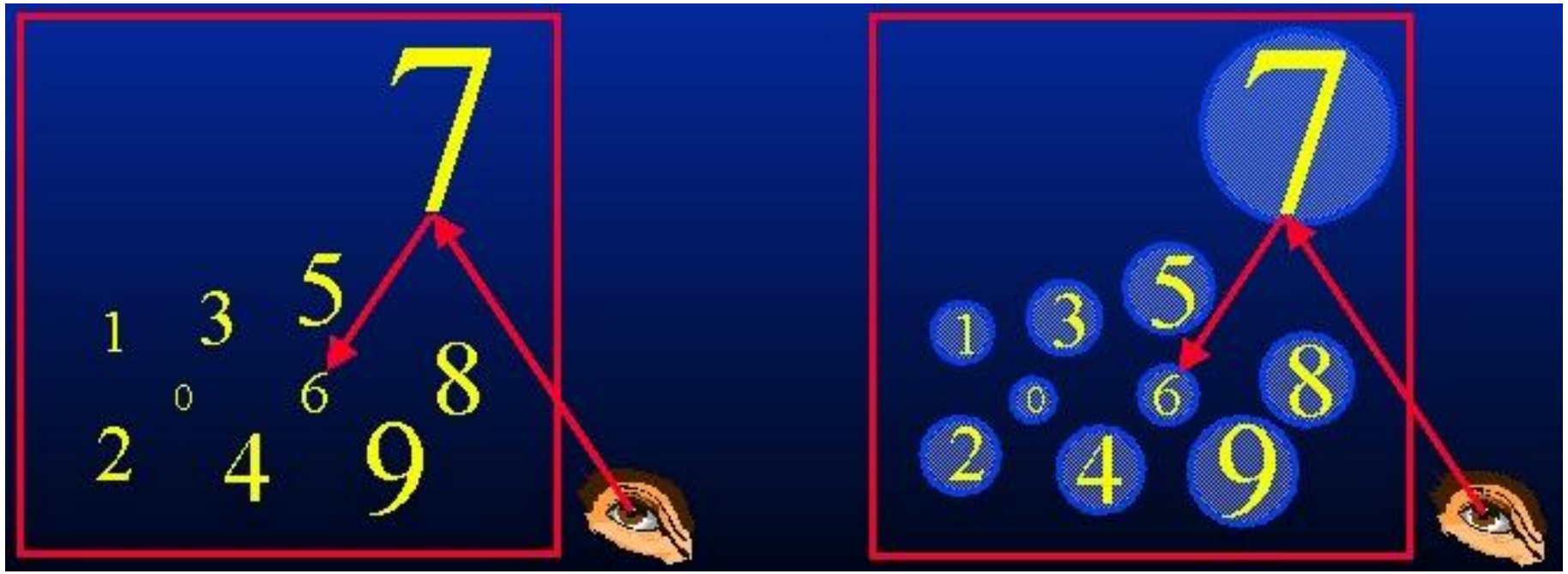
Adaptive Subsampling





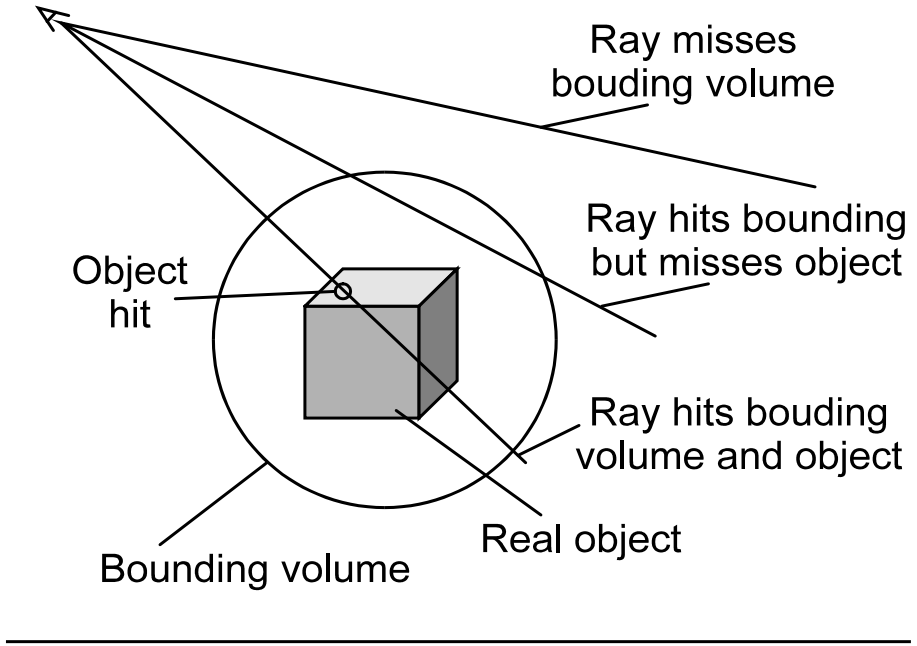


Bounding Volumes (Obalová tělesa)

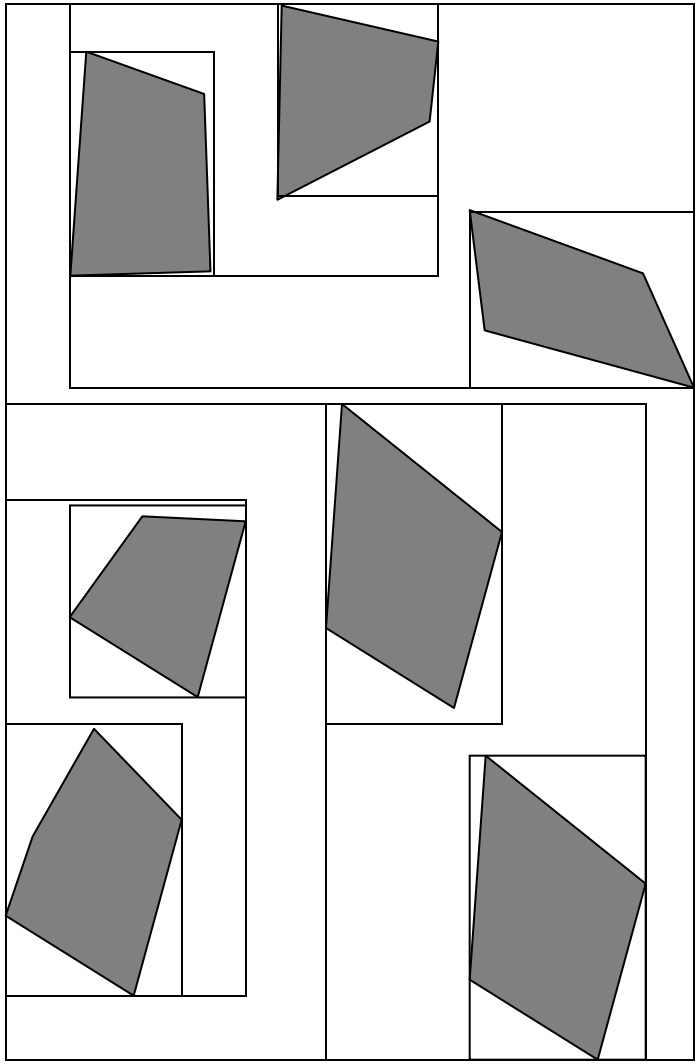


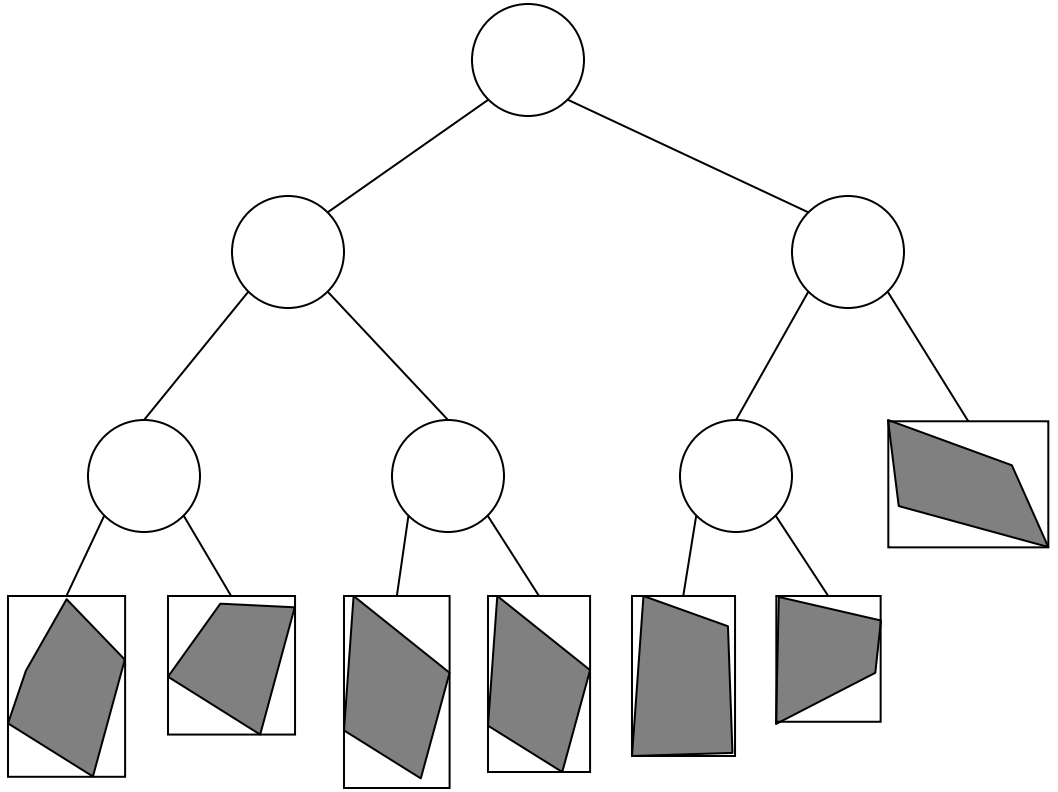
Bounding Volumes

- Ruční zadání obalových těles
- Koule, AABB, OBB, Slabs
- Hierarchical bounding volumes
- Speedup >10x



Bounding Volume Hierarchy

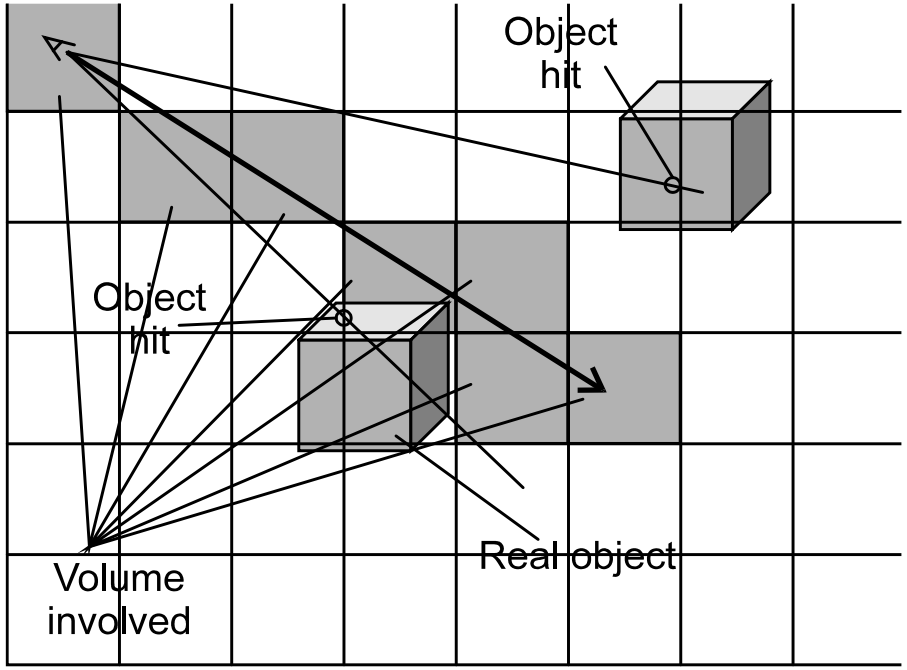




Space subdivision (Dělení prostoru)

SaarCOR.avi

- Uniformní mřížka (2D, 3D)
  - + Jednoduché, rychlé pořízení
  - + Jednoduchý průchod (jak?)
  - Není adaptivní (plýtvá)
- Octree
  - + Jednoduché pořízení
  - + Adaptivní
  - Nesnadné procházení
  - Adaptivní pouze v měřítku
- k-D tree (http://www.rolemaker.dk/nonRoleMaker/uni/algogem/kdtree.htm)
  - + Adaptivnější než Octree
  - Složitý průchod, struktura
- BSP-tree (http://symbolcraft.com/graphics/bsp/index.html)
  - + Extrémně adaptivní
  - Složité
- Speedup >10x



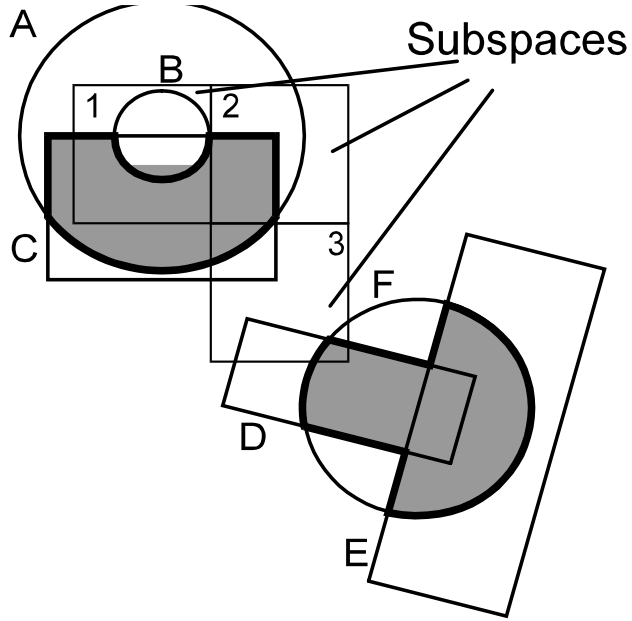
Graphics Hardware 2004

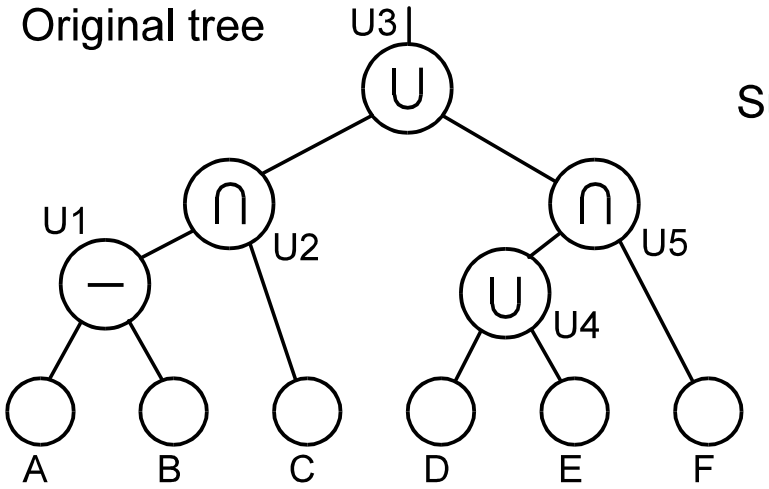
Realtime Ray Tracing of Dynamic Scenes on an FPGA Chip

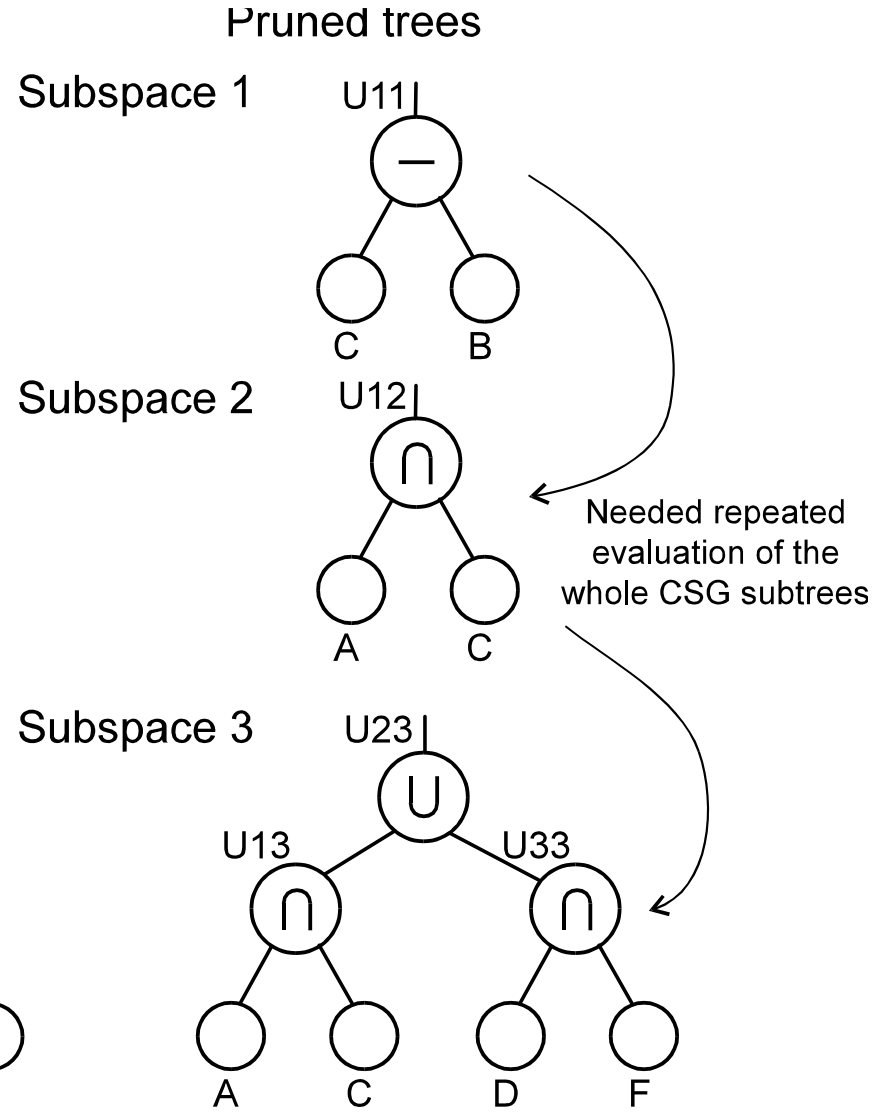
Joerg Schmittler, Sven Woop, Daniel Wagner,
Wolfgang Paul, and Philipp Slusallek

Computer Science, Saarland
University, Germany

CSG tree pruning (prořezávání stromu)



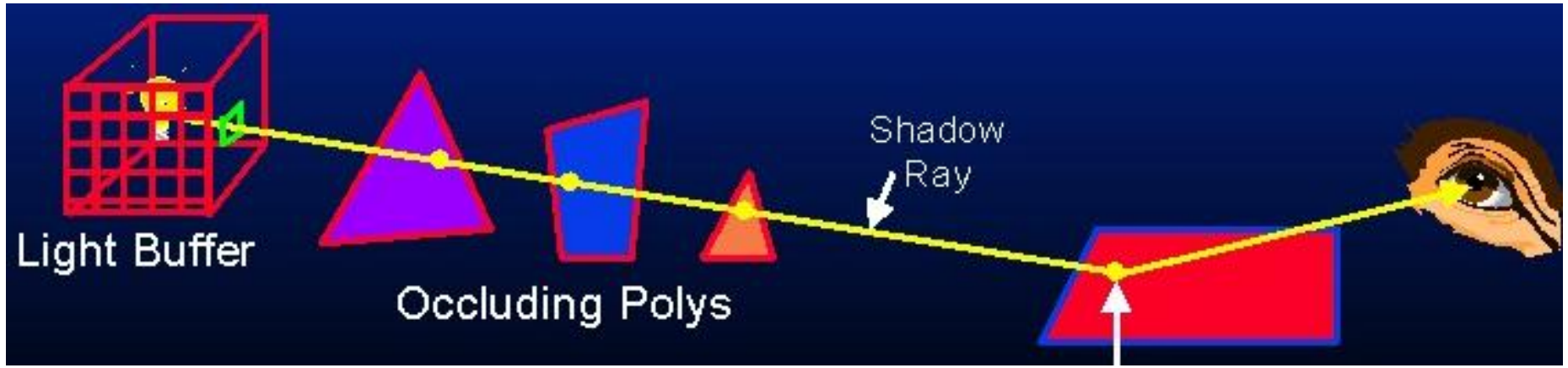




Light buffer

NatureStillSucks
RPU.avi

- S využitím akcelerátoru à la OpenGL
- Stínová mapa – obsah paměti hloubky



RPU:
A Programmable Ray Processing Unit for
Realtime Ray Tracing

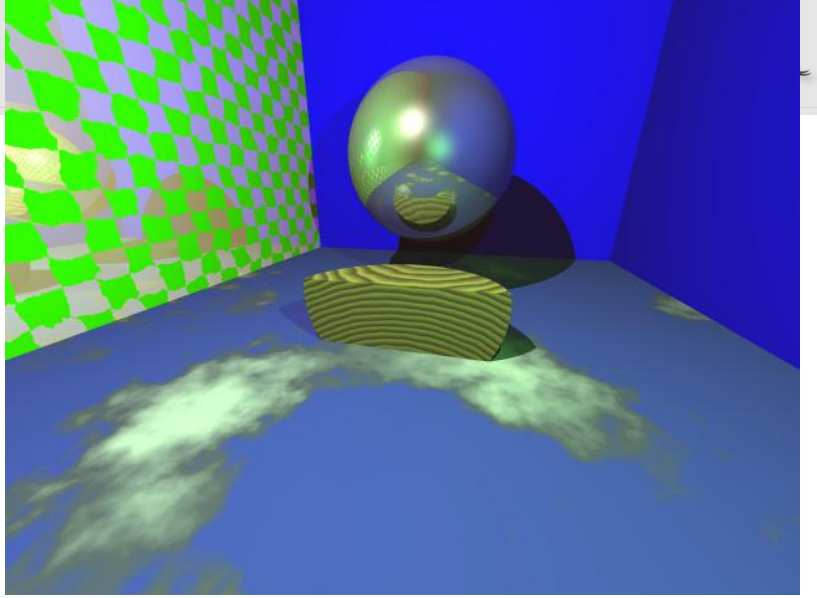
Sven Woop Jörg Schmittler Philipp Slusallek

Computer Graphics Lab,
Saarland University, Germany

Siggraph 2005

Aurelius

- Stručný Ray-Tracer (1500 řádků)
- Co umí?
  - Primární paprsky
  - Stínové paprsky (bodové světlo)
  - Odražené paprsky
  - Procedurální textury
  - Kouli, kvádr, rovinu
  - Adaptivní supersampling (proti aliasu)
- Proč vznikl?
  - Aby byl stručný



Aurelius je stručný

```
#include "Vector.h"

class Ray {
  Vector a;
  Vector p;
public:
  Ray(const Vector &start, const Vector &direction)
  {
     a = start; p = direction;
  }
  inline const Vector &Start() const { return a; }
  inline const Vector &Direction() const { return p; }
  inline Vector GetPoint(double t) const { return a + t*p; }
  void ShiftStart(double shiftby = 1e-6) { a += shiftby*p;  }
};
```

Aurelius je stručný

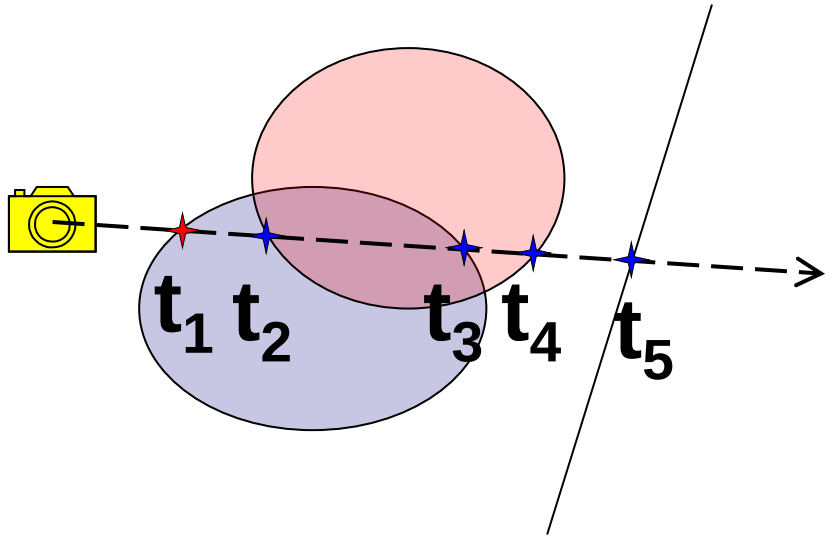
```
class Sphere : public Shape {
  Vector c; double r;
public:
  Sphere(const Vector &centre, double radius) { c = centre; r = radius; }
  virtual Intersection IntersectFirst(const Ray &ray) {
    const Vector &a = ray.Start();  const Vector &p = ray.Direction();
    double aa = p*p;   double bb = 2* (p * (a - c));
    double cc = (a-c)*(a-c) - r*r;  double D = bb*bb - 4*aa*cc;
    if (D > 0) {
      double sD = sqrt(D);
      double t1 = (-bb - sD) / (2*aa);
      if (t1 > 0) return Intersection(Intersection::ikInto, this, t1);
      double t2 = (-bb + sD) / (2*aa);
      if (t2 > 0) return Intersection(Intersection::ikOutfrom, this, t2);
    }
    return Intersection(Intersection::ikNone);
  }
  virtual Vector GetNormal(const Vector &position) {
    Vector n = position - c;  n.Normalize();
    return n;
  }
};
```

Aurelius je stručný

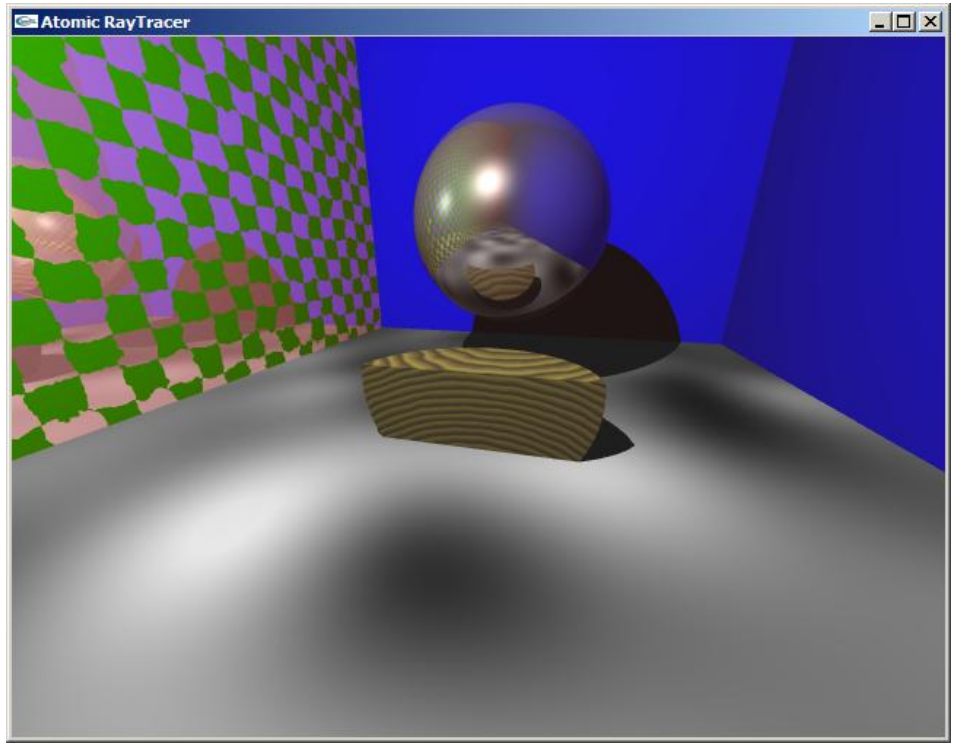
```
class Wood : public Material {
  PhongMap pmap;
  Vector centre, direction;
  float k;
public:
  Wood(PhongMap map,
       const Vector &centre = Vector(0, 0, 0),
       const Vector &dir = Vector(1, 0, 0), float coef = 1.0)
      : pmap(map) {
    this->centre = centre; k = coef;
    direction = dir; direction.Normalize();
  }
  virtual const PhongInfo GetPhongInfo(const Vector &position) {
    Vector a = position - centre;
    Vector dist = a - (a*direction) * direction;
    return pmap.Eval(fmod(k * dist.Length(), 1.0));
  }
};
```

Aurelius je modulární

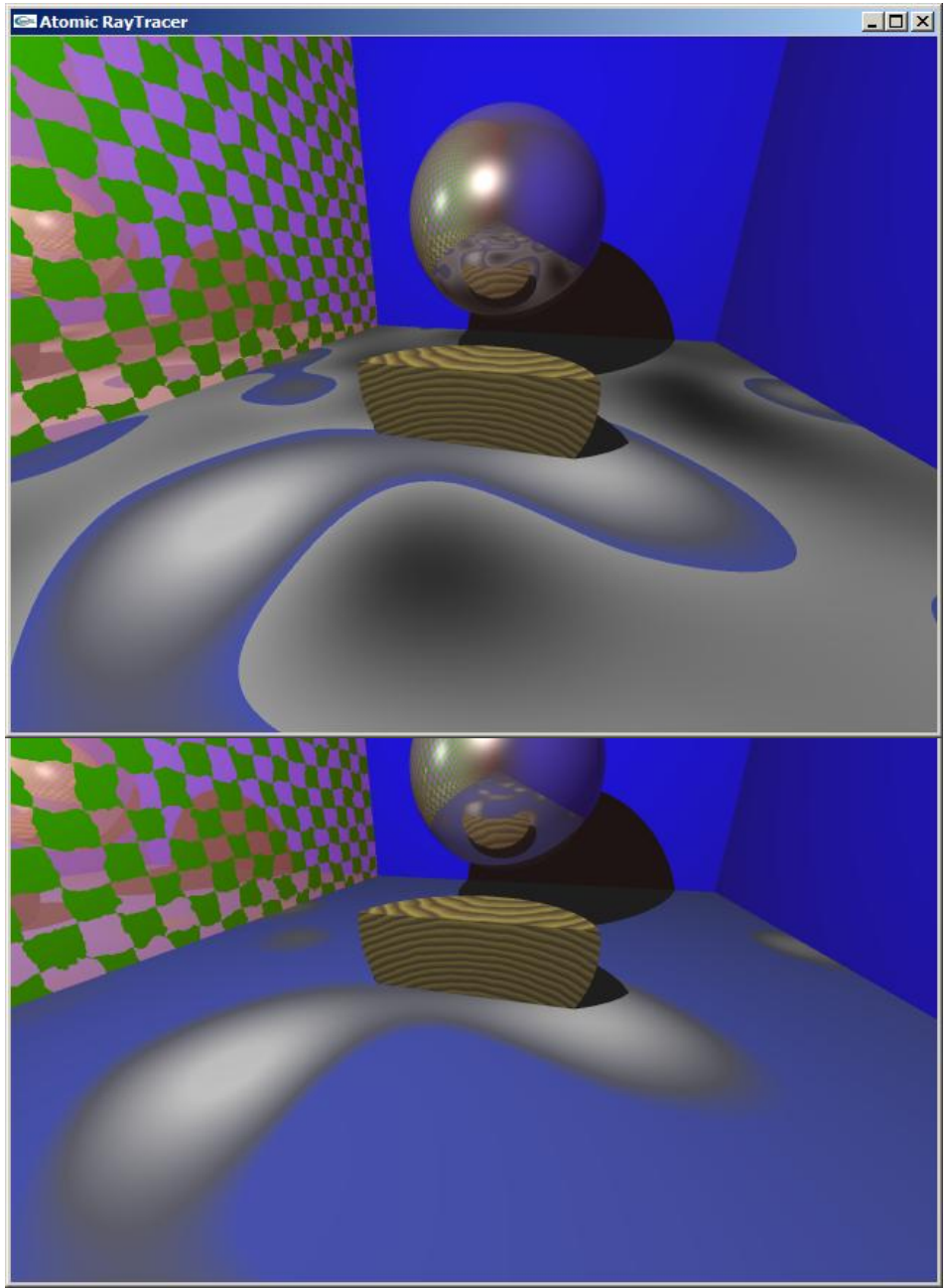
- Matematika – Vector, Ray
- Geometrie – Sphere, Plane, Cylinder
- CSG
  - Intervalové množiny
- Šumy (vnoise, pnoise, turbulence)



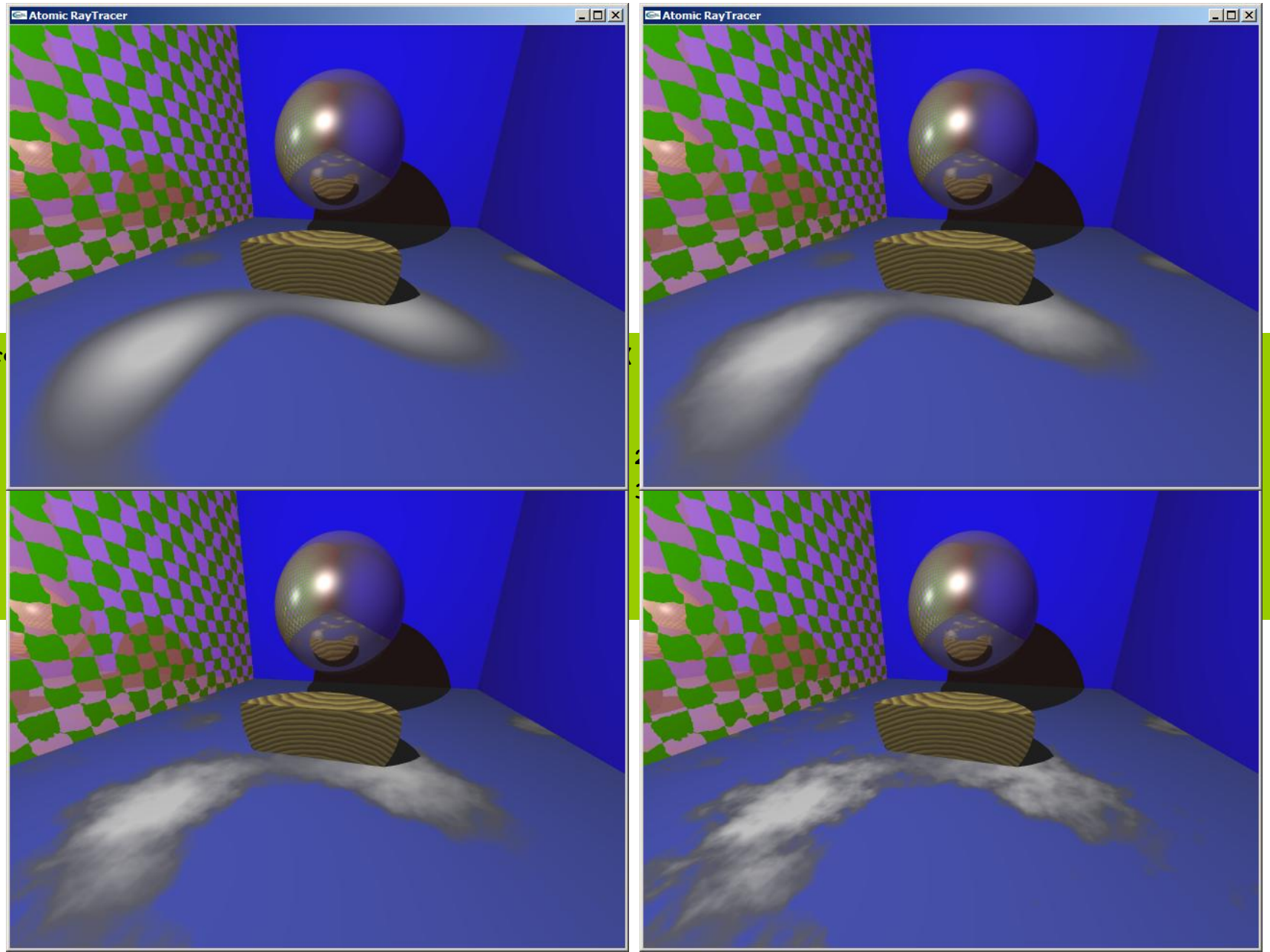
Šumy - vnoise




```
<< ColorMap::Segment(.6, .7,
     Color(0.2, 0.25, 0.7),
     Color(0.3, 0.3,0.36))
<< ColorMap::Segment(.7, 1.,
     Color(0.3, 0.3, 0.36),
     Color(0.8, 0.8, 0.8))
```



Šumy - turbulence



Seznam tříd Aurelia k říjnu 2006

- `class AtomicRayTracer` – celý ray-tracer

- `class Color` – barva (rgb)
- `class ColorMap` – barevná mapa (0.0-1.0)
  - `class ColorMap::Segment` – segment mapy

- `class Image` – výsledný obrázek (pole class Color)

- `class Camera` – kamera (pozice, pohled, promítání)

- `class Light` – obecné světlo
- `class PointLight : public Light` – bodové světlo
- `class LightSet` – množina světel (třeba všechna)

Seznam tříd Aurelia k říjnu 2006

- `class PhongInfo` – diffuse, specular, ambient, shininess, reflectance
- `class PhongMap` – jako ColorMap, ale celé PhongInfo

- `class Material` – obecný materiál
- `class PhongMaterial : public Material` – jako glMaterial

- `class NoiseDisplace : public Mat.` – modifikuje mater. šumem

- `class Checker : public Material` – šachovnice
- `class Marble : public Material` – mramor (rovinné vrstvy)
- `class Wood : public Material` – dřevo (koaxiální válce)
- `class Clouds : public Material` – mraky (ValueNoise)

Seznam tříd Aurelia k říjnu 2006

- `struct Vector` – 3D vektor
- `class Ray` – paprsek

- `class Noise` – obecný šum
- `class ImprovedNoise : public Noise` – VG noise
- `class ValueNoise : public Noise` – V noise

- `template <class N> class PerlinNoise : public Noise`
  - příklad `new PerlinNoise<ImprovedNoise>`

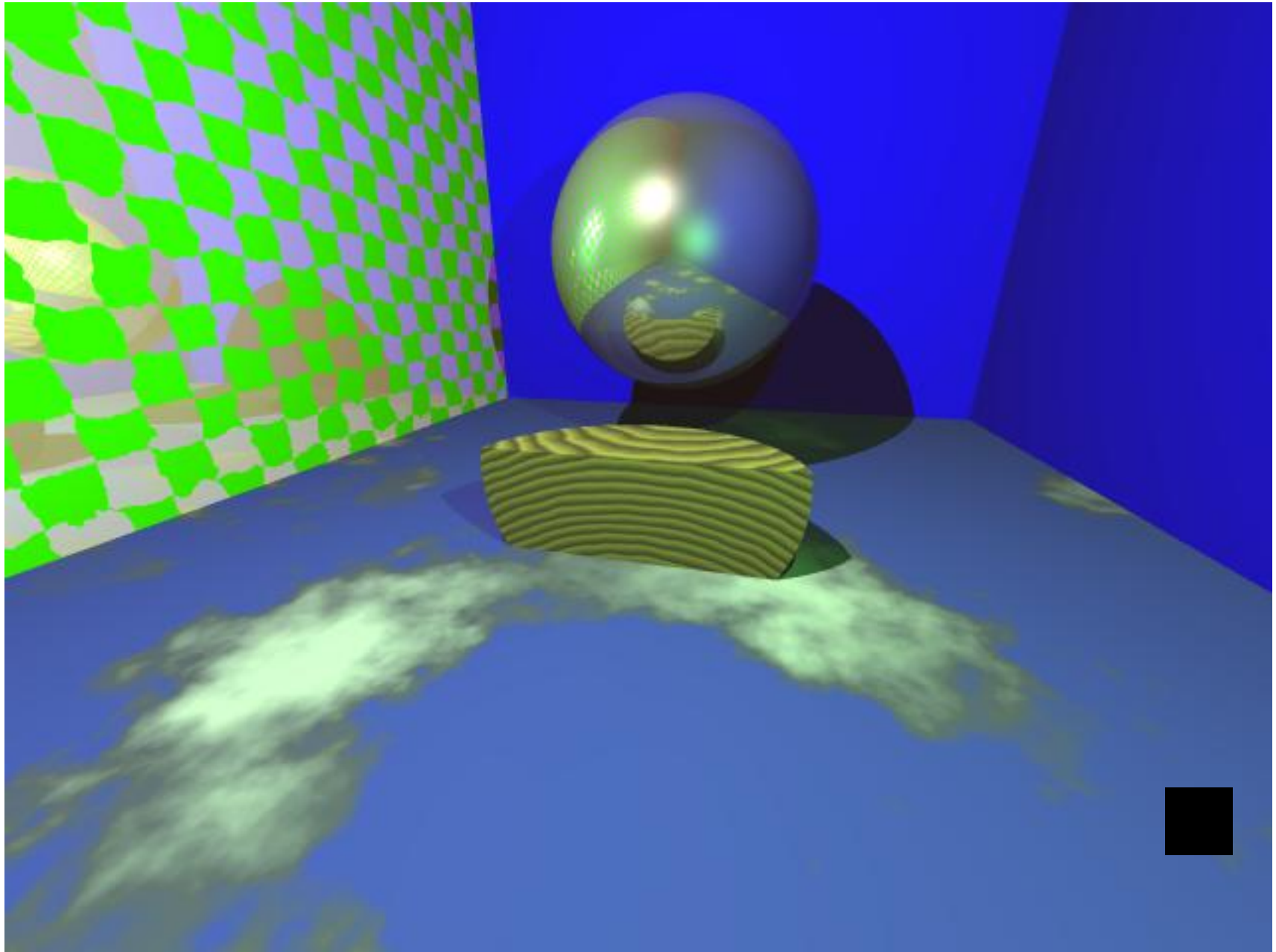
Seznam tříd Aurelia k říjnu 2006

- `class Shape` – prostorový tvar (průsečík, intervaly)
- `class Sphere : public Shape` – koule
- `class Cylinder : public Shape` – válec
- `class Plane : public Shape` – rovina

- `class Intersection : public Shape` – CSG, průnik
- `class Inverse : public Shape` – CSG, doplněk ve vesmíru

- `class Object` – objekt (tvar + materiál)
- `class ObjectGroup : public Object` – skupina objektů
- `class ShapeObject : public Object` – primitivní objekt

Obrázek



Suma sumárum

- Sledování paprsku – Ray tracing
  - Zpětné rekurzivní
- Kamera, průmětna
- Paprsky – primární, sekundární, stínové
- Osvětlení – Phongův model
  - Procedurální texturování
- Urychlování
  - adaptive subsampling, bounding volumes, space subdivision, CSG tree pruning
- Aurelius